

**252-0027**

**Einführung in die Programmierung  
Übungen**

**Woche 7: Hoare-Logik, Problem Solving**

**Joran Bühler  
Departement Informatik  
ETH Zürich**

# Bonus

Implementieren Sie die Methode `countAssimilated` in der Klasse `Matrix`, welche eine  $m \times n$ -Matrix  $A$  (mit  $m > 2, n > 2$ ) von `int`-Werten als Input akzeptiert und einen `int`-Wert zurückgibt. Für alle Elemente  $a_{i,j}$  von  $A$  gilt  $a_{i,j} \neq 0$ .

Diese Methode soll die Anzahl der *an ihre Umgebung angepassten* Elemente in der Input-Matrix  $A$  berechnen und zurückgeben. Ein Element  $a_{i,j}$  von  $A$  gilt als angepasst, wenn die Summe seiner direkten Nachbarn (vertikal, horizontal und diagonal, soweit in der Matrix  $A$  definiert) ohne Rest durch  $a_{i,j}$  teilbar ist. Das heisst, dass ein Element  $a_{i,j}$  genau dann angepasst ist, wenn

$$(a_{i-1,j-1} + a_{i-1,j} + a_{i-1,j+1} + a_{i,j-1} + a_{i,j+1} + a_{i+1,j-1} + a_{i+1,j} + a_{i+1,j+1}) \% a_{i,j} == 0$$

gilt. Wenn das Element  $a_{x,y}$  nicht existiert (da  $x < 0$  oder  $x \geq m, y < 0, y \geq n$ ), dann wird der Wert in der Berechnung durch 0 ersetzt.

Sie können davon ausgehen, dass der Parameter `matrix` nie den Wert `null` hat und nie auf eine  $m \times n$  Matrix mit  $m \leq 2$  oder  $n \leq 2$  verweist. Sie können auch davon ausgehen, dass die Summe der Nachbarn sich ohne Overflow oder Underflow berechnen lässt.

Hier sind ein paar Beispielaufrufe der Methode sowie das erwartete Ergebnis (im "test"-Ordner finden Sie die entsprechenden JUnit-Tests):

1. `countAssimilated([[10, 10, 10], [10, 10, 10], [10, 10, 10]]): 9`
2. `countAssimilated([[5, 10, 3], [6, 9, 6], [3, 3, 15]]): 4`
3. `countAssimilated([[4, 7, 13], [-2, -12, 32], [20, 15, -8], [17, 3, 1111]]): 3`
4. `countAssimilated([[1, 2, 3, 4, 5, 6], [7, 8, 9, 10, 11, 12], [13, 14, 15, 16, 17, 18], [19, 20, 21, 22, 23, 24], [25, 26, 27, 28, 29, 30]]): 15`

# **Weakest Precondition II**

# Stärkere und schwächere Aussagen

- Wir können *stärkere* und *schwächere* Aussagen unterscheiden
  - Wenn  $P_1 \Rightarrow P_2$  gilt, dann ist  $P_1$  stärker als  $P_2$  (und  $P_2$  schwächer als  $P_1$ )
  - Falls  $P_1$  wahr ist, ist  $P_2$  auch wahr.
  - Bsp:  $P_1: x > 4$  (Stärker)    $P_2: x > 0$  (Schwächer)
- **Starke Aussage:** Eine Aussage oder Voraussetzung, die **mehr Einschränkungen** auferlegt. Sie gilt in weniger Fällen.
- **Schwache Aussage:** Eine Aussage oder Voraussetzung, die **weniger Einschränkungen** auferlegt. Sie ist allgemeiner.

# Preconditions / Postconditions

- Die **schwächste Precondition** ist die schwächste Precondition, für die noch die Postcondition erfüllt ist.
  - «Was dürfen wir der Funktion für Werte geben, damit wir einen erwünschten Output haben?»
- Die **stärkste Postcondition** ist die stärkste Postcondition, welche noch aus der Precondition folgt.
  - «Was kann alles als Resultat kommen?»

# Weakest Precondition

- Die **schwächste Vorbedingung (weakest precondition)** ist die schwächste Vorbedingung, die die Postcondition impliziert.
  - Falls die Postcondition  $\{ \text{true} \}$  ist, so ist  $\{ \text{true} \}$  die schwächste Vorbedingung. Alles impliziert die Postcondition, also insbesondere auch die schwächste Bedingung  $\text{true}$ .
  - Falls die Postcondition  $\{ \text{false} \}$  ist, so ist  $\{ \text{false} \}$  die schwächste Vorbedingung. Nur  $\{ \text{false} \}$  impliziert die Postcondition, dementsprechend ist es die schwächste (und einzige) Vorbedingung.
- Die vorgestellten Regeln fürs Rückwärtsschliessen ergeben direkt die schwächsten Vorbedingungen.

# If-Anweisungen und Aussagen

- **Ziel:** Regel für Tripel  $\{P\} \text{ if } (b) S_1 \text{ else } S_2 \{Q\}$

- **Beobachtungen**

- Ausführung von  $S_1$  wenn  $b$  hält
- Ausführung von  $S_2$  wenn  $\neg b$  hält
- $P$  hält in beiden Fällen vor der Ausführung
- $Q$  muss in beiden Fällen nachher gelten

```
 $\{P\}$   
    if (b) {  
         $S_1$ ;  
    } else {  
         $S_2$ ;  
    }  
 $\{Q\}$ 
```

# Hoare-Logik-Regel für if-Anweisungen

- Das Tripel  $\{P\} \text{ if } (b) S_1 \text{ else } S_2 \{Q\}$  ist gültig, genau dann wenn
  1.  $\{P \wedge b\} S_1 \{Q\}$  gültig ist
  2.  $\{P \wedge \neg b\} S_2 \{Q\}$  gültig ist



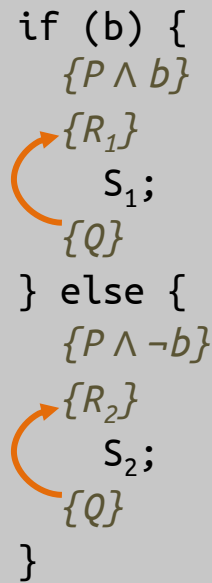
# Vorgehen für if-Anweisungen

- **Situation:** Entscheide, ob  $\{P\}$  if (b)  $S_1$  else  $S_2$   $\{Q\}$  gültig ist

- **Empfohlenes Vorgehen:**

1. Wende bekannte Regeln wie rechts gezeigt an (auf  $S_1$  und  $S_2$ )
2. Zeige notwendige Implikationen
  1.  $P \wedge b \Rightarrow R_1$
  2.  $P \wedge \neg b \Rightarrow R_2$

```
if (b) {  
    {P ∧ b}  
    {R1}  
    S1;  
    {Q}  
} else {  
    {P ∧ ¬b}  
    {R2}  
    S2;  
    {Q}  
}
```

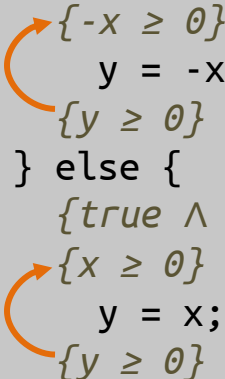


# Beispiel 1: Gültiges Tripel

Zu zeigen: Gültigkeit von  $\{true\} \text{ if } (x < 0) \{ y = -x; \} \text{ else } \{ y = x; \} \{y \geq 0\}$

1. Regeln anwenden:

```
if (x < 0) {  
    {true ∧ x < 0}  
    {-x ≥ 0}  
    y = -x;  
    {y ≥ 0}  
} else {  
    {true ∧ ¬(x < 0)}  
    {x ≥ 0}  
    y = x;  
    {y ≥ 0}  
}
```



2. Implikationen zeigen:

1.  $(x < 0) \Rightarrow (-x \geq 0)$  ✓
2.  $(x \geq 0) \Rightarrow (x \geq 0)$  ✓

# Beispiel 2: Ungültiges Tripel

Geändert

Zu zeigen: Gültigkeit von  $\{true\} \text{ if } (x < 0) \{ y = -x; \} \text{ else } \{ y = x; \} \{y > 0\}$

1. Regeln anwenden:

```
if (x < 0) {  
  {true ∧ x < 0}  
  {-x > 0}  
  y = -x;  
  {y > 0}  
} else {  
  {true ∧ ¬(x < 0)}  
  {x > 0}  
  y = x;  
  {y > 0}  
}
```

Entsprechende  
Unterschiede

2. Implikationen zeigen:

1.  $(x < 0) \Rightarrow (-x > 0)$  ✓
2.  $(x \geq 0) \Rightarrow (x > 0)$  ✗

Hält nicht mehr  
(Gegenbeispiel:  $x == 0$ )

# Beispiel 3: Ohne else-Zweig

Zu zeigen: Gültigkeit von  $\{x == y \wedge x \neq 0\} \text{ if } (a > 1) \{ a = a * x; \} \{a \neq y\}$

1. Regeln anwenden:

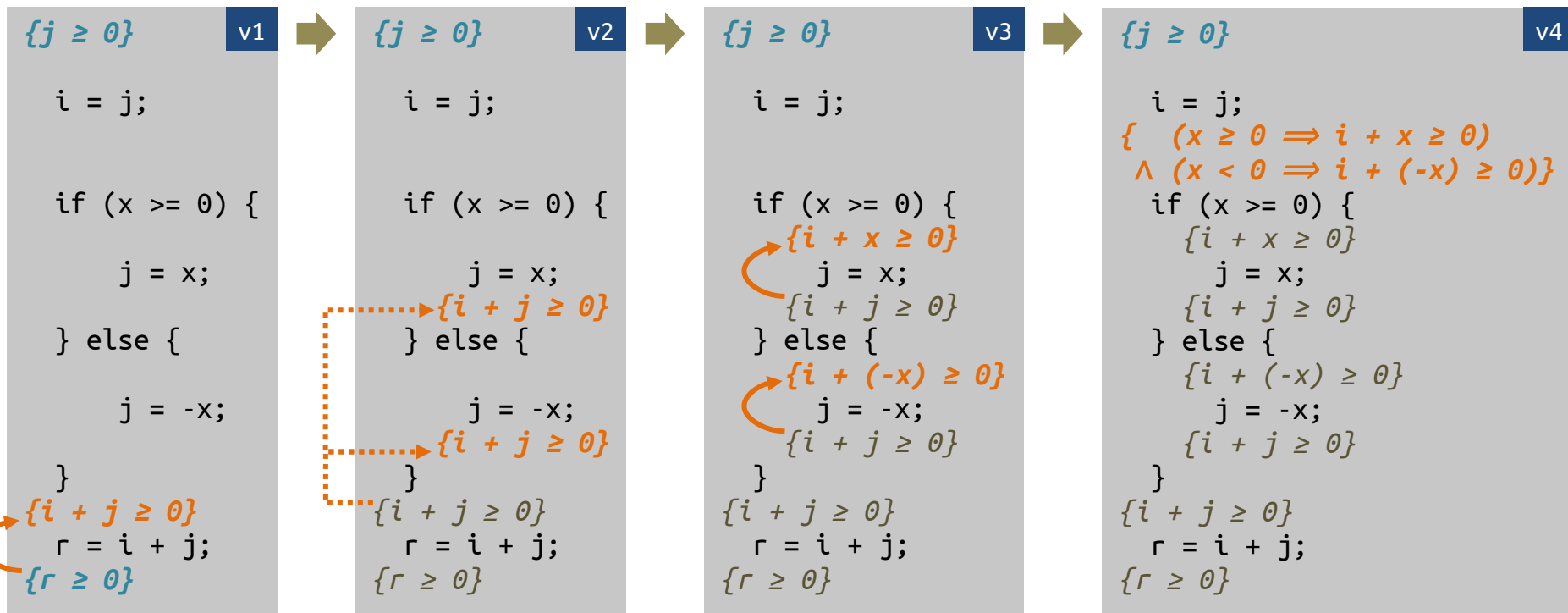
```
if (a > 1) {  
    {x == y ∧ x != 0 ∧ a > 1}  
    {a * x ≠ y}  
    a = a * x;  
    {a ≠ y}  
} else {  
    {x == y ∧ x != 0 ∧ ¬(a > 1)}  
    {a ≠ y}  
}
```

Leerer else-Block entspricht  
fehlendem else-Block

2. Implikationen zeigen:

1.   $(x == y \wedge x \neq 0 \wedge a > 1) \Rightarrow (a * x \neq y)$
2.   $(x == y \wedge x \neq 0 \wedge a \leq 1) \Rightarrow (a \neq y)$

# Beispiel Schritt für Schritt



v4



{j ≥ 0}

v5

```

{ (x ≥ 0 ⇒ j + x ≥ 0)
  ∧ (x < 0 ⇒ j + (-x) ≥ 0) }
  i = j;
{ (x ≥ 0 ⇒ i + x ≥ 0)
  ∧ (x < 0 ⇒ i + (-x) ≥ 0) }
  if (x >= 0) {
    {i + x ≥ 0}
    j = x;
    {i + j ≥ 0}
  } else {
    {i + (-x) ≥ 0}
    j = -x;
    {i + j ≥ 0}
  }
  {i + j ≥ 0}
  r = i + j;
  {r ≥ 0}

```

Dann noch zu zeigen:

$$(j \geq 0)$$

$$\Rightarrow$$

$$((x \geq 0 \Rightarrow j + x \geq 0) \wedge (x < 0 \Rightarrow j + (-x) \geq 0))$$

Fallunterscheidung (zwei Konjunkte):

1. Wenn  $j \geq 0$  und  $x \geq 0$ , dann  $j + x \geq 0$  ✓
2. Wenn  $j \geq 0$  und  $x < 0$ , dann  $j + (-x) \geq 0$  ✓

# **Weakest Precondition mit Verzweigung**

# Schwächste Vorbedingung - Beispiel

$(x \geq y \wedge x \geq x) \parallel (\neg x \geq y \wedge y \geq x)$   
if else

{ }

if (x >= y){

y = x

}

{y >= x}

$x \geq y$  |  $\neg(x \geq y)$   
1 | 1  
 $x \geq x$  |  
 $y \geq x$  |  $y \geq x$



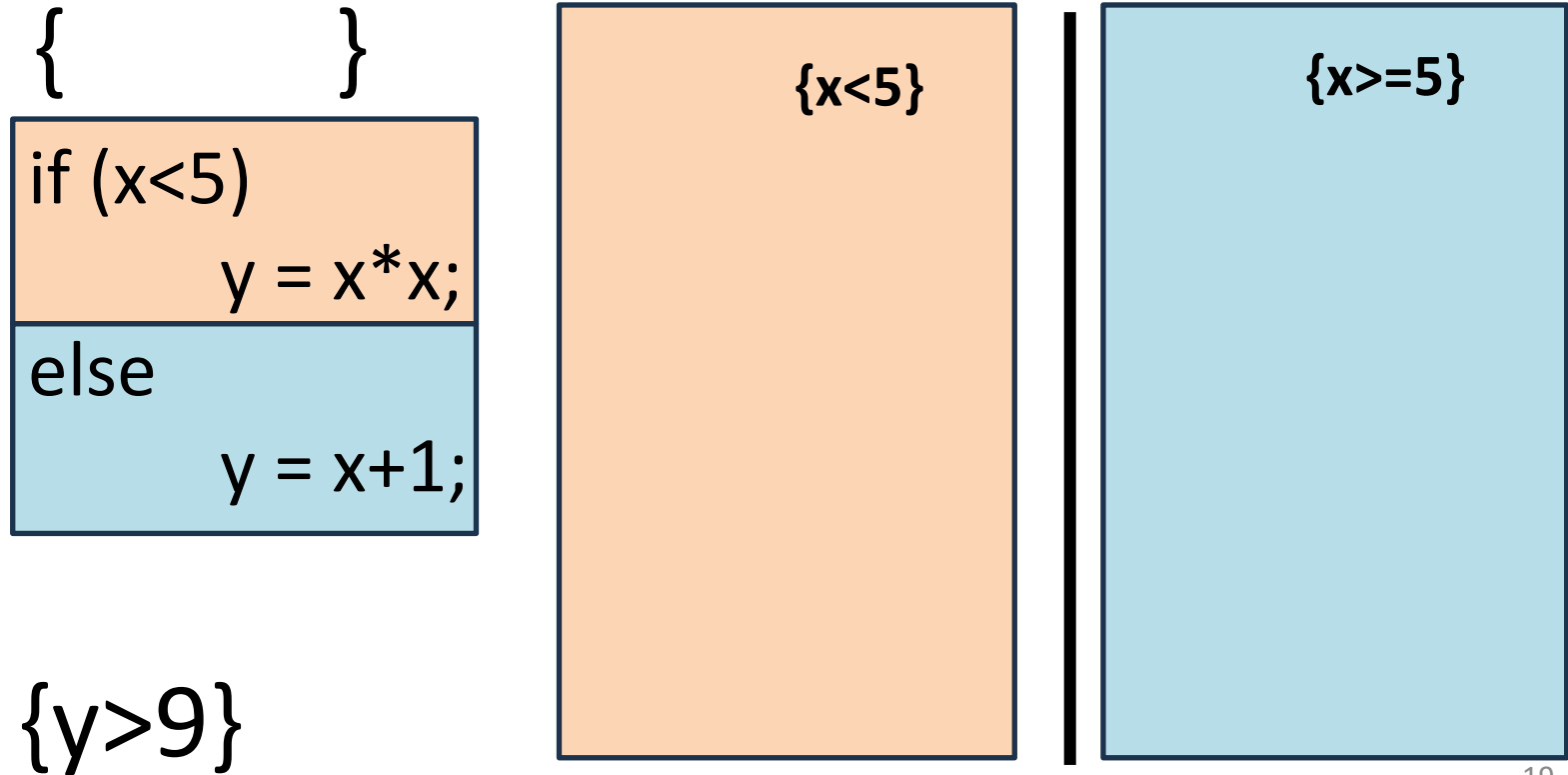
# Weakest Precondition – Mit Verzweigung

```
{      }
```

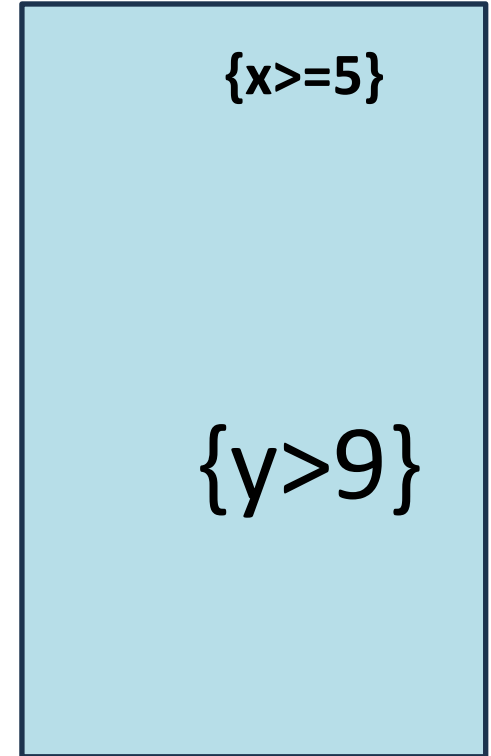
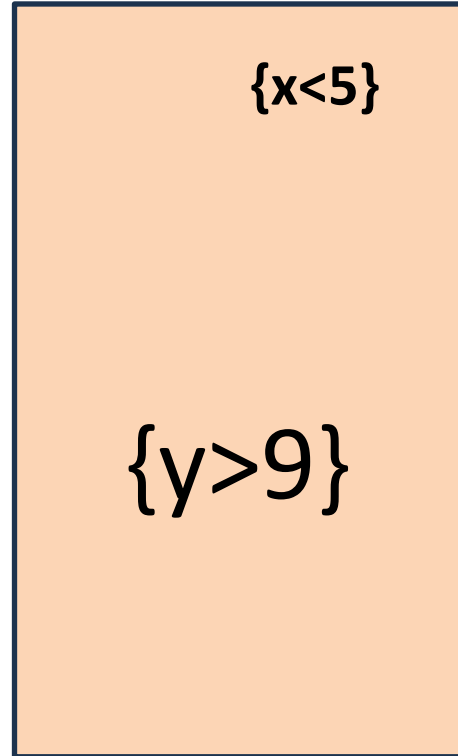
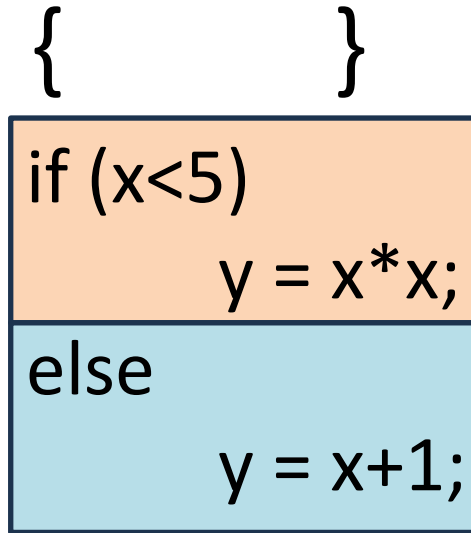
```
if (x<5) {  
    y = x*x;  
} else {  
    y = x+1;  
}
```

```
{y>9}
```

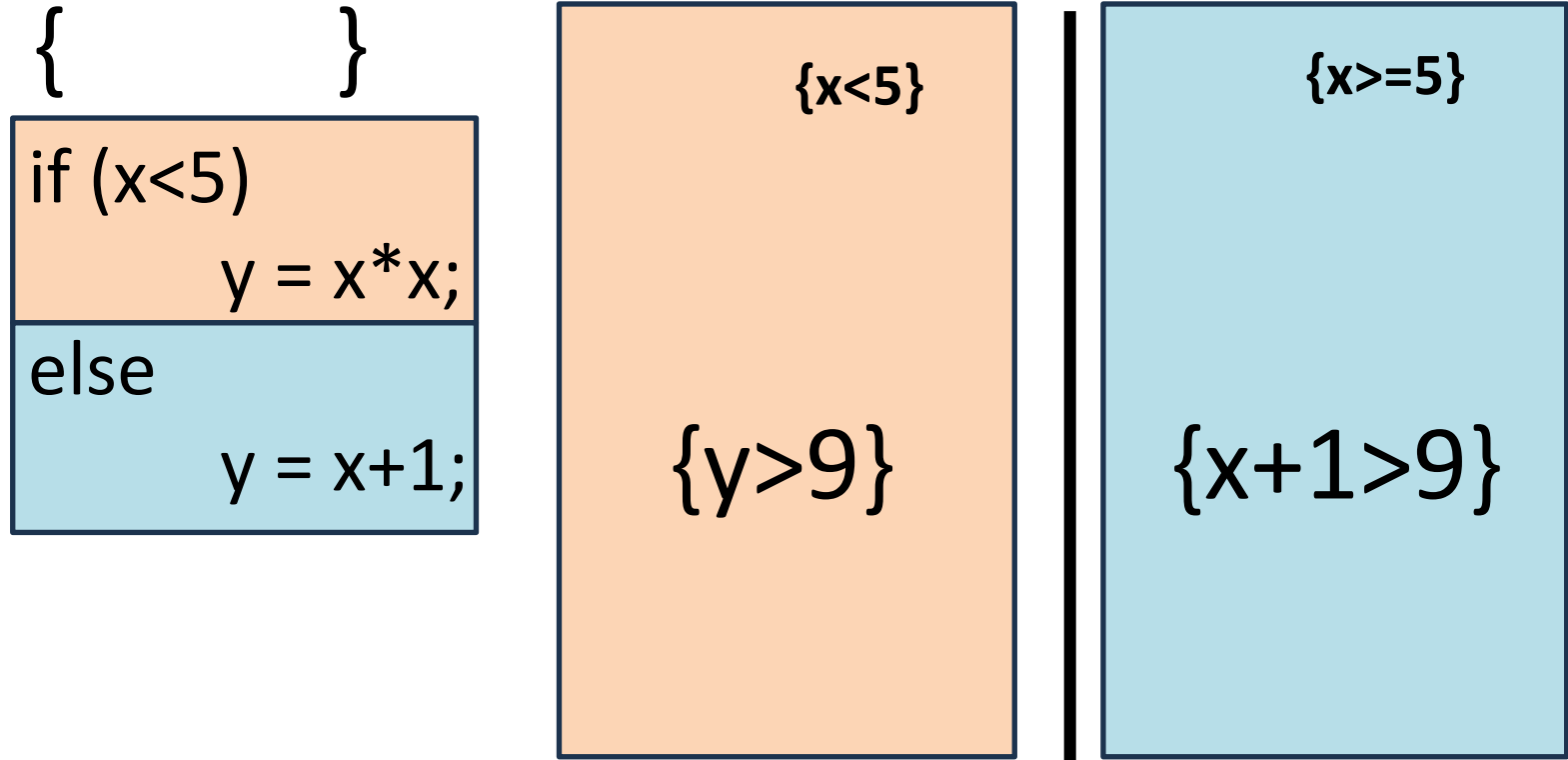
# Weakest Precondition – Mit Verzweigung



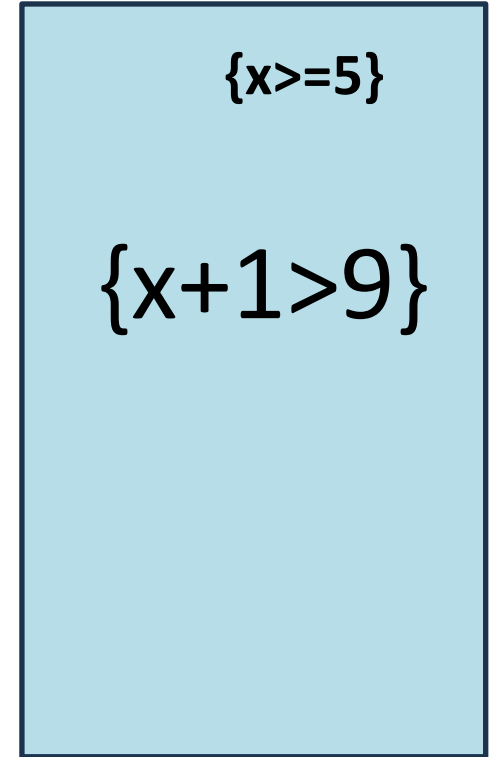
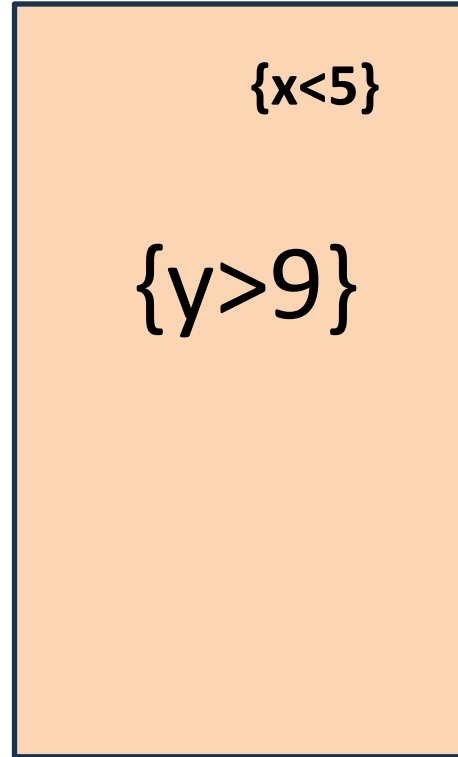
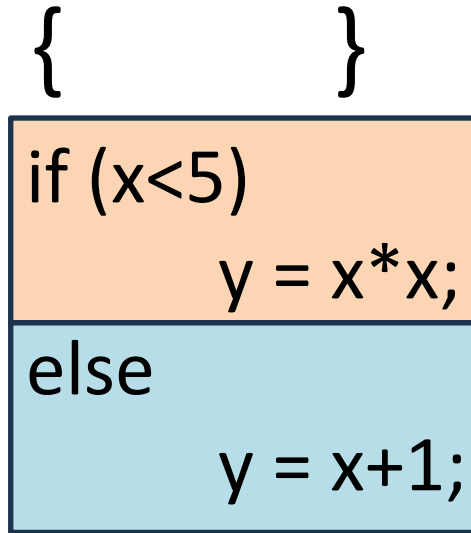
# Weakest Precondition – Mit Verzweigung



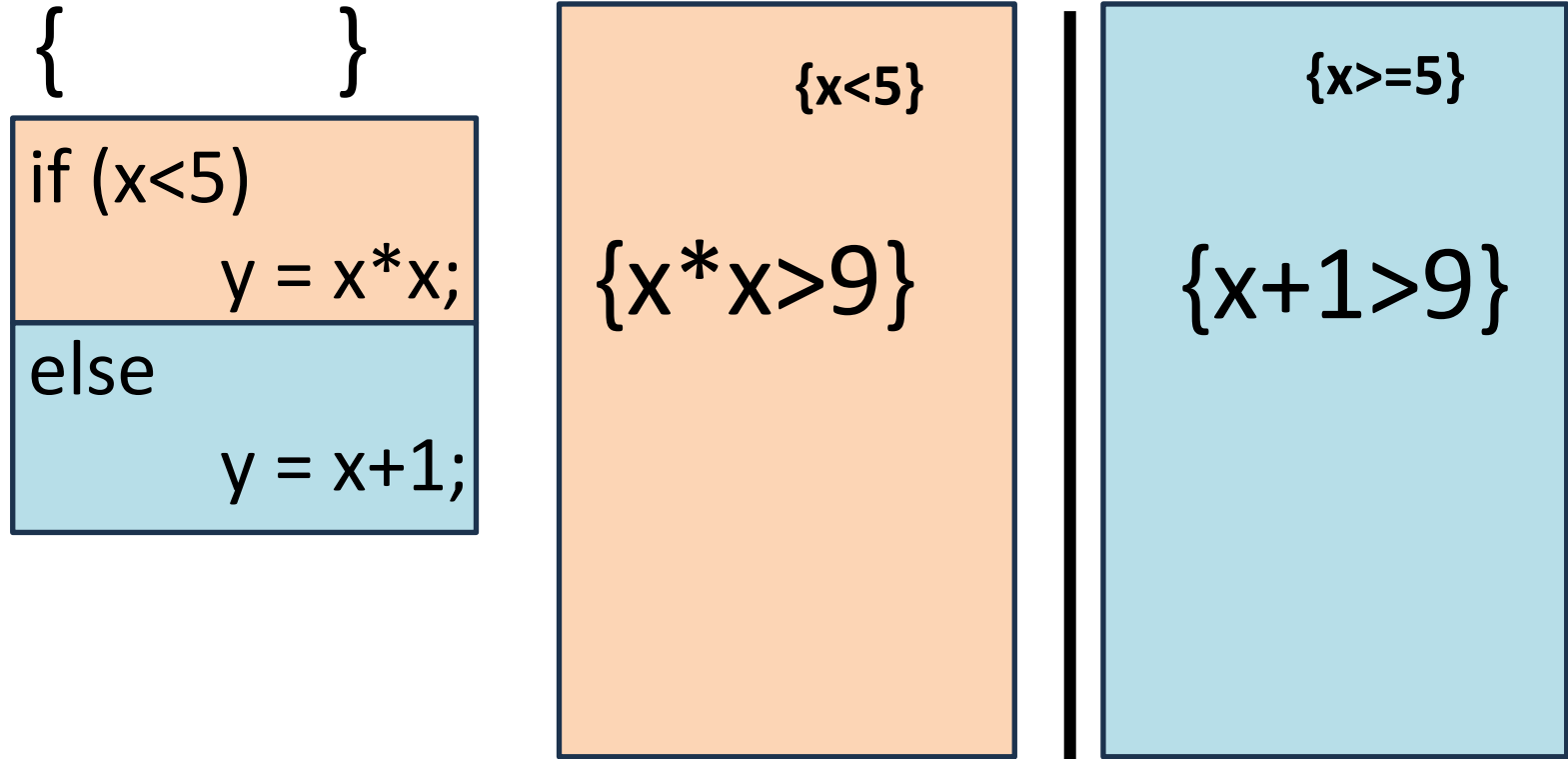
# Weakest Precondition – Mit Verzweigung



# Weakest Precondition – Mit Verzweigung



# Weakest Precondition – Mit Verzweigung



$x == 4 \vee x < -3$   $x > 8$   
 $\{(x < 5 \ \&\& \ x * x > 9) \vee (x \geq 5 \ \&\& \ x + 1 > 9)\}$

{ }

if ( $x < 5$ )  
     $y = x * x;$   
else  
     $y = x + 1;$

$\{x < 5\}$

$\{x * x > 9\}$

$\{x \geq 5\}$

$\{x + 1 > 9\}$

# Hoare Triple



# Hoare Tripel – Prüfungsbeispiel HS18

```
{ b > c }  
  if (x > b) {  
    a = x;  
  } else {  
    a = b;  
  }  
{ a > c }
```

# Hoare Tripel – Prüfungsbeispiel HS18

```
{ true }  
  if (x > y) {  
    y = x;  
  } else {  
    y = -x;  
  }  
{ y >= x }
```

# Hoare Tripel – Prüfungsbeispiel HS18

$$\{ x > 0 \}$$
$$y = x * x;$$
$$z = y / 2;$$
$$\{ z > 0 \}$$

# Hoare Logik FAQ

- Müssen wir unsere Ausdrücke vereinfachen?
  - Solange die Lösung **logisch äquivalent** ist, ist sie korrekt.
- Müssen wir unsere Herleitung zeigen?
  - Solange dies **nicht explizit gefragt** wird, ist dies nicht nötig.
- Müssen Gegenbeispiele alle Variablen enthalten, wenn nur eine davon relevant ist?
  - Wenn z.B.  $y \neq 0$  immer bedeutet, dass aus einer Precondition nicht die Postcondition folgt, dann ist dies ausreichend.

# Permutation Check

# Beispielsaufgabe

Gegeben sind die Integer Arrays  $s$  und  $t$ , überprüfe, ob  $t$  eine Permutation von  $s$  ist.

**Lösung 1:** Generiere alle Permutationen von  $s$  und schaue ob  $t$  eine davon ist.

# Beispielsaufgabe

Gegeben sind die Integer Arrays  $s$  und  $t$ , überprüfe, ob  $t$  eine Permutation von  $s$  ist.

**Lösung 1:** Generiere alle Permutationen von  $s$  und schaue ob  $t$  eine davon ist.

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
    at TestingStuff/module.Permutation.permute(Permutation.java:16)
    at TestingStuff/module.Permutation.permute(Permutation.java:23)
    at TestingStuff/module.Permutation.permute(Permutation.java:23)
    at TestingStuff/module.Permutation.permute(Permutation.java:23)
    at TestingStuff/module.Permutation.permute(Permutation.java:23)
    at TestingStuff/module.Permutation.permute(Permutation.java:23)
```

# Out of Memory?

- Java erhält für das Ausführen eines Programms eine **beschränkte** Menge Arbeitsspeicher (Memory).
- In der vorherigen Lösung generieren wir  **$O(n!)$  Permutation**, welche alle Speicher benötigen.
- Irgendwann hat es **nicht mehr genug Speicher** und das Programm terminiert mit einem Fehler (Error).



# Beispielsaufgabe

Gegeben sind die Integer Arrays  $s$  und  $t$ , überprüfe, ob  $t$  eine Permutation von  $s$  ist.

**Lösung 2:** Generiere alle Permutationen von  $s$  und schaue ob  $t$  eine davon ist, während wir die Permutationen generieren.



# Beispielsaufgabe

Gegeben sind die Integer Arrays  $s$  und  $t$ , überprüfe, ob  $t$  eine Permutation von  $s$  ist.

**Lösung 3:** Prüfe ob  $s$  und  $t$  gleich lang sind und ob sie die Gleichen Elemente enthalten.

# Kahoot

- <https://create.kahoot.it/share/u7-eprog-recursion-wpc-and-hoare-tripel/26632932-a421-44f1-a64a-7107e1818286>

0 /

# Aufgabe 5: Levels

## ■ Bonus

**Achtung:** Diese Aufgabe gibt Bonuspunkte (siehe “Leistungskontrolle” im [www.vvz.ethz.ch](http://www.vvz.ethz.ch)). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich [die allgemeinen Regeln](#).

Implementieren Sie die Methode `sumLevel(int[] [] matrix, int level)` in der Klasse `Levels`, die für eine quadratische Matrix (Parameter `matrix`) und eine angegebene „Level“-Nummer (Parameter `level`) die Summe der Elemente dieses Levels berechnet.

Ein Level ist definiert durch die Schichten der Matrix, die von innen nach aussen gezählt werden. Level 0 entspricht dem Zentrum der Matrix. Höhere Level beschreiben jeweils die nächsten, äusseren Ränder der Matrix.

In Abbildung 2 finden Sie zwei Beispiele für Matrizen und ihre Levels. In Abbildung 2a beispielsweise besteht das innerste Level, Level 0, lediglich aus einem Element. Abbildung 2b zeigt eine Matrix mit drei Levels: Der äussere Rand der Matrix ist Level 2, der mittlere Rand Level 1 und Level 0 besteht aus den innersten vier Elementen.

|               |               |               |
|---------------|---------------|---------------|
| 1<br>Level: 1 | 2<br>Level: 1 | 3<br>Level: 1 |
| 4<br>Level: 1 | 5<br>Level: 0 | 6<br>Level: 1 |
| 7<br>Level: 1 | 8<br>Level: 1 | 9<br>Level: 1 |

(a)  $3 \times 3$ -Matrix

|                |                |                |                 |                 |                 |
|----------------|----------------|----------------|-----------------|-----------------|-----------------|
| 10<br>Level: 2 | 20<br>Level: 2 | 30<br>Level: 2 | 40<br>Level: 2  | 50<br>Level: 2  | 60<br>Level: 2  |
| 15<br>Level: 2 | 25<br>Level: 1 | 35<br>Level: 1 | 45<br>Level: 1  | 55<br>Level: 1  | 65<br>Level: 2  |
| 70<br>Level: 2 | 80<br>Level: 1 | 90<br>Level: 0 | 100<br>Level: 0 | 110<br>Level: 1 | 120<br>Level: 2 |
| 5<br>Level: 2  | 10<br>Level: 1 | 15<br>Level: 0 | 20<br>Level: 0  | 25<br>Level: 1  | 30<br>Level: 2  |
| 1<br>Level: 2  | 2<br>Level: 1  | 3<br>Level: 1  | 4<br>Level: 1   | 5<br>Level: 1   | 6<br>Level: 2   |
| 99<br>Level: 2 | 88<br>Level: 2 | 77<br>Level: 2 | 66<br>Level: 2  | 55<br>Level: 2  | 44<br>Level: 2  |

(b)  $6 \times 6$ -Matrix

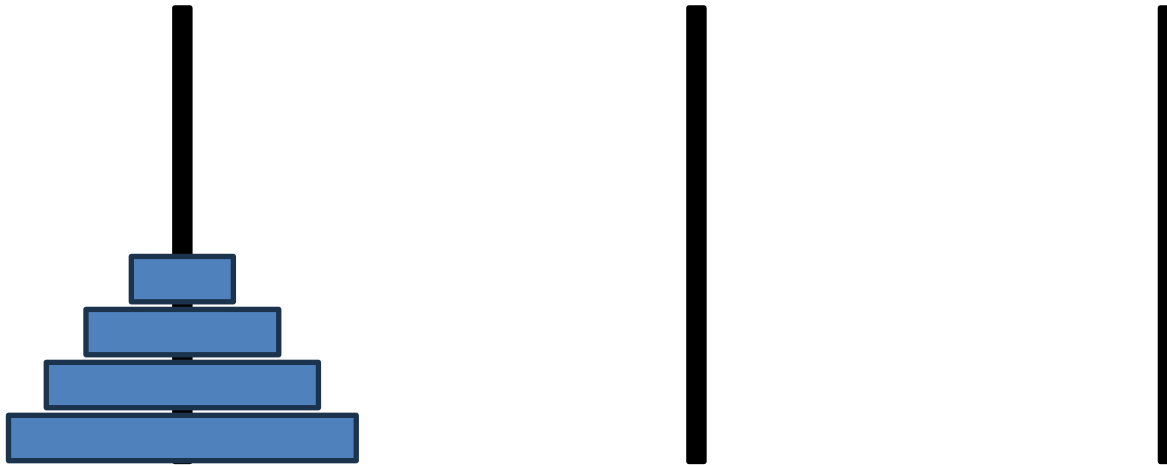
Abbildung 2: Beispiele für gültige Matrizen und ihre Levels

# **Towers of Hanoi**

# Towers of Hanoi - Beschreibung

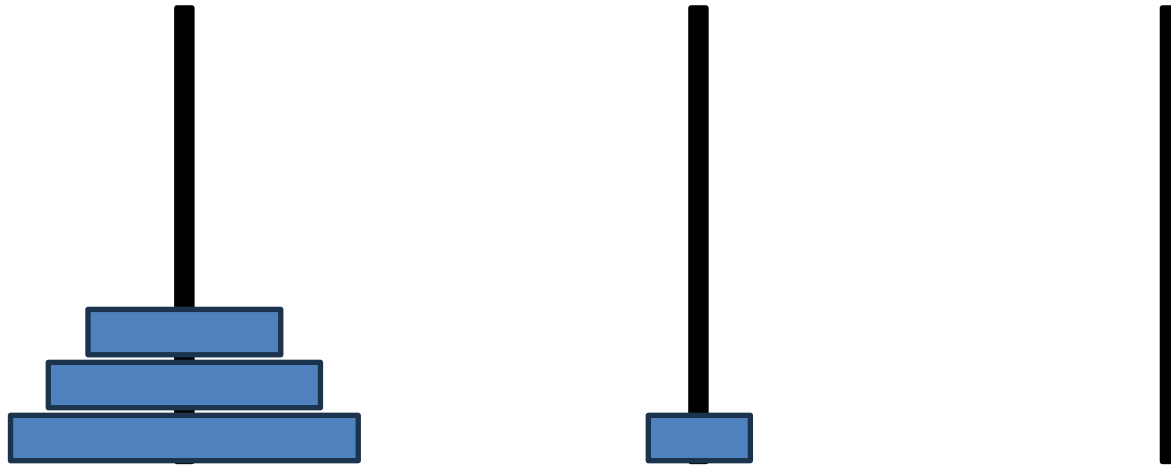
Das Türme von Hanoi-Problem besteht aus drei Stäben und einer Anzahl von unterschiedlich grossen Scheiben, die zu Beginn auf einem Stab gestapelt sind, wobei die grösste Scheibe unten und die kleinste oben liegt. Ziel ist es, alle Scheiben von einem Ausgangsstab auf einen Zielstab zu bewegen, wobei nur eine Scheibe auf einmal bewegt werden darf und niemals eine grössere Scheibe auf eine kleinere gelegt werden darf.

# Towers of Hanoi - Beispiel

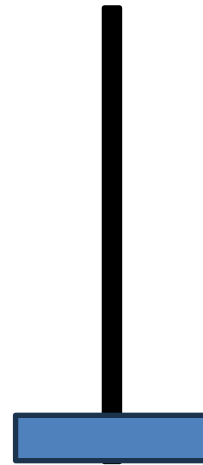
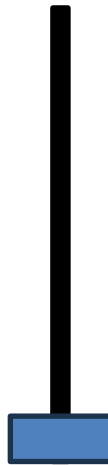
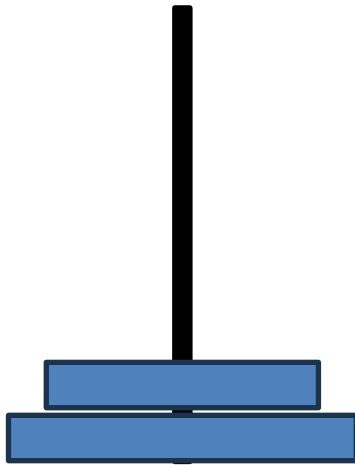




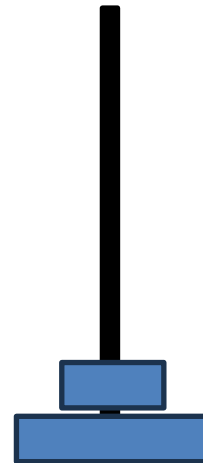
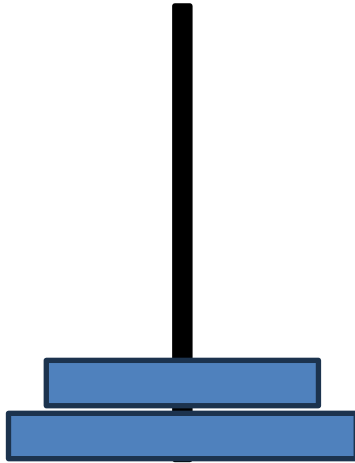
# Towers of Hanoi - Beispiel



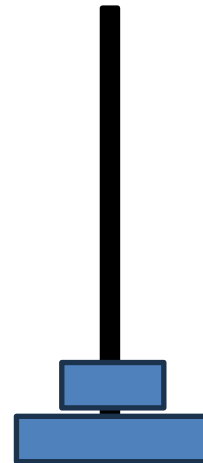
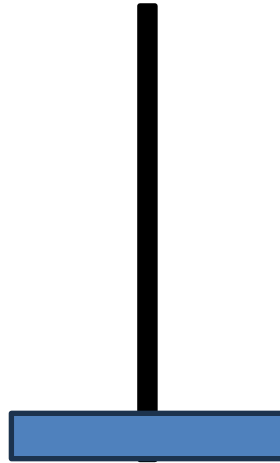
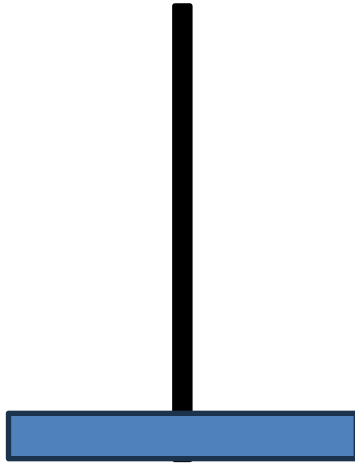
# Towers of Hanoi - Beispiel



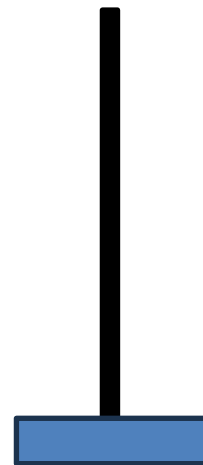
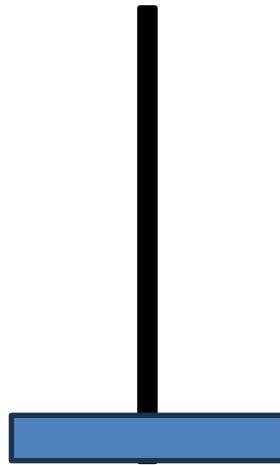
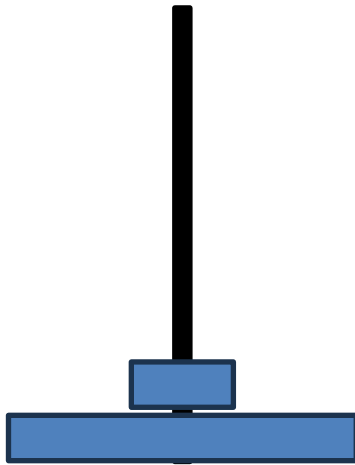
# Towers of Hanoi - Beispiel



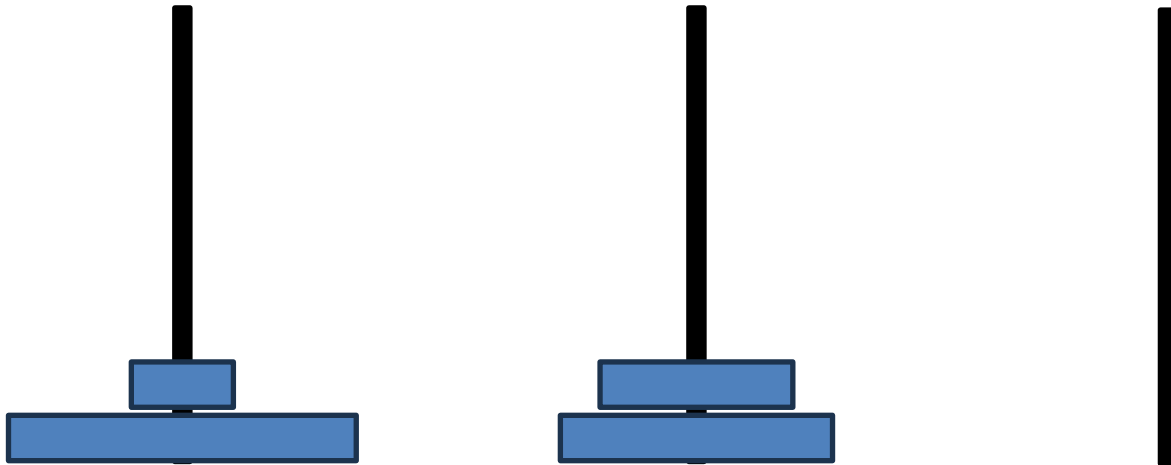
# Towers of Hanoi - Beispiel



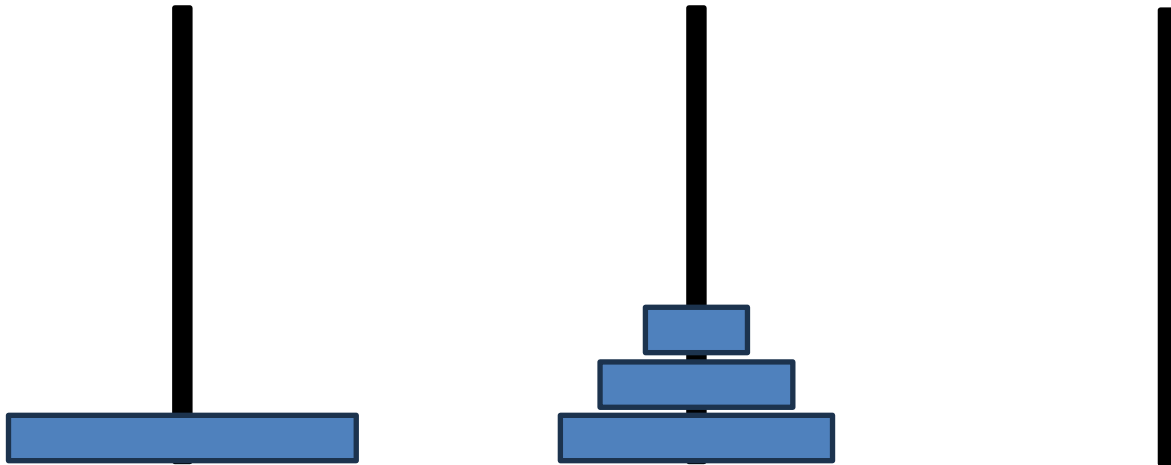
# Towers of Hanoi - Beispiel



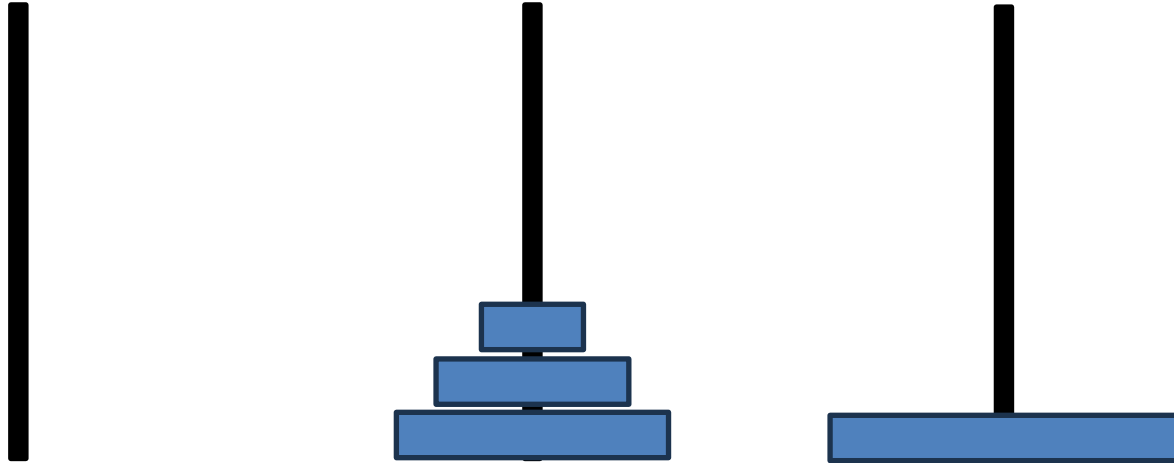
# Towers of Hanoi - Beispiel



# Towers of Hanoi - Beispiel

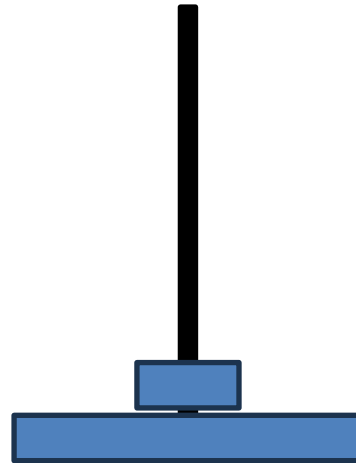
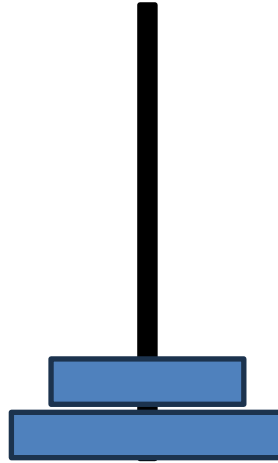


# Towers of Hanoi - Beispiel

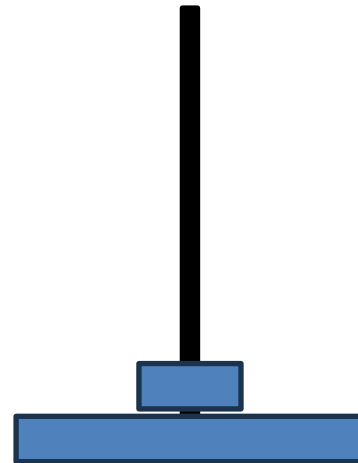
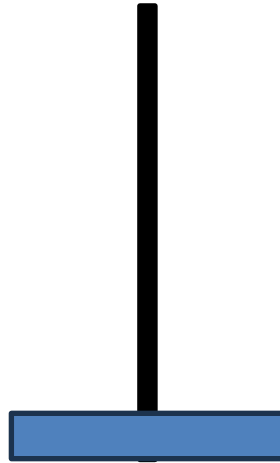
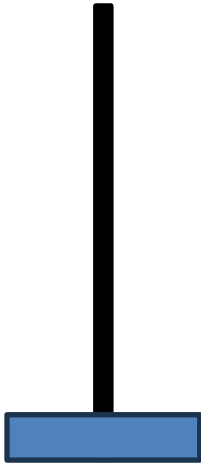




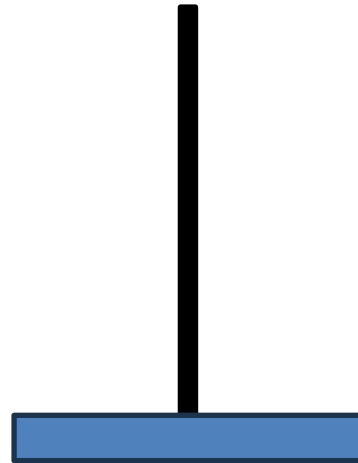
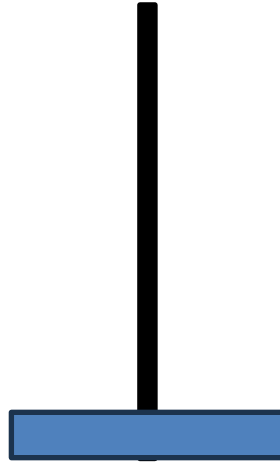
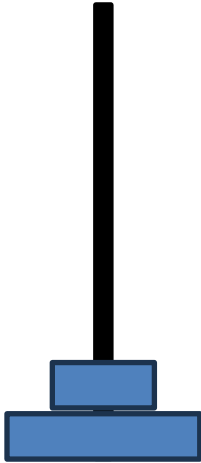
# Towers of Hanoi - Beispiel



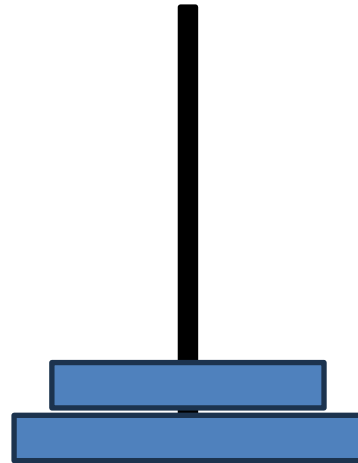
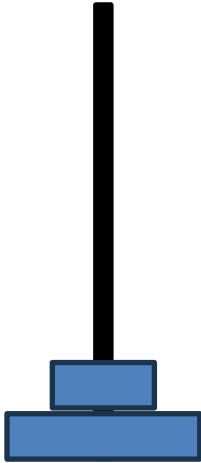
# Towers of Hanoi - Beispiel



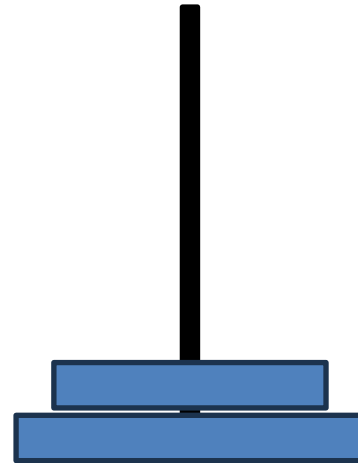
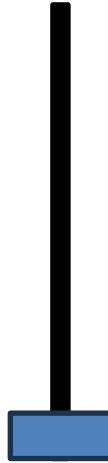
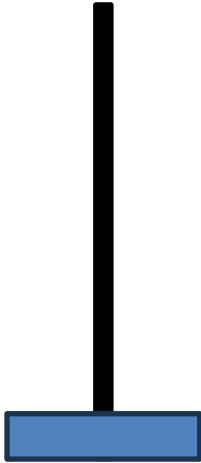
# Towers of Hanoi - Beispiel



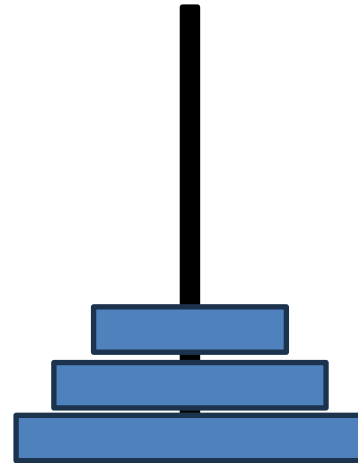
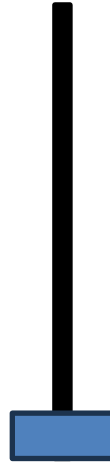
# Towers of Hanoi - Beispiel



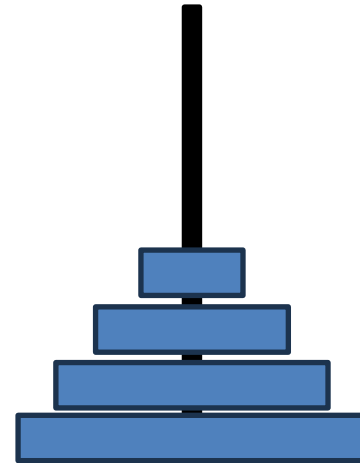
# Towers of Hanoi - Beispiel

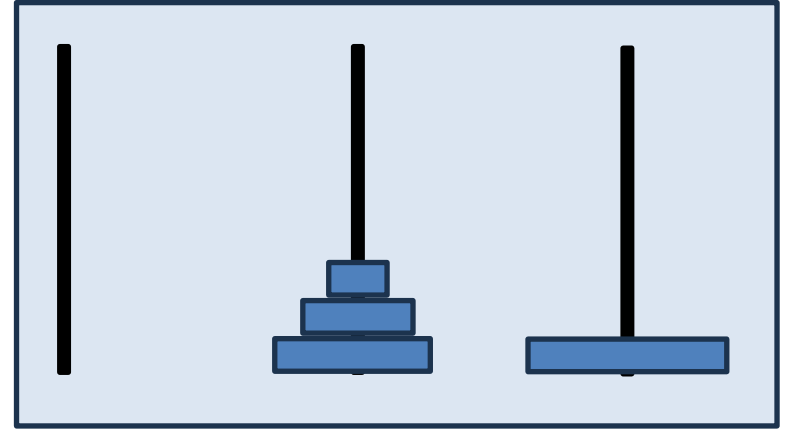
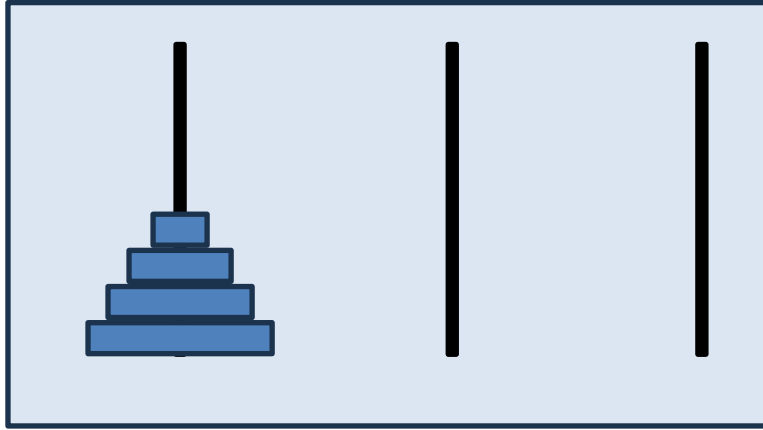


# Towers of Hanoi - Beispiel



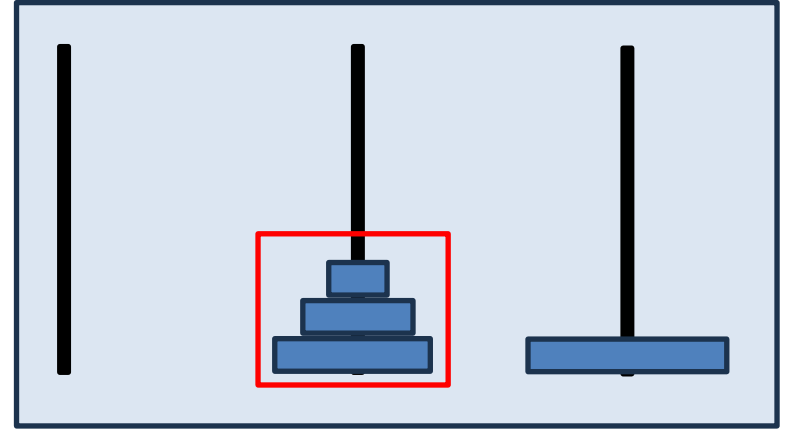
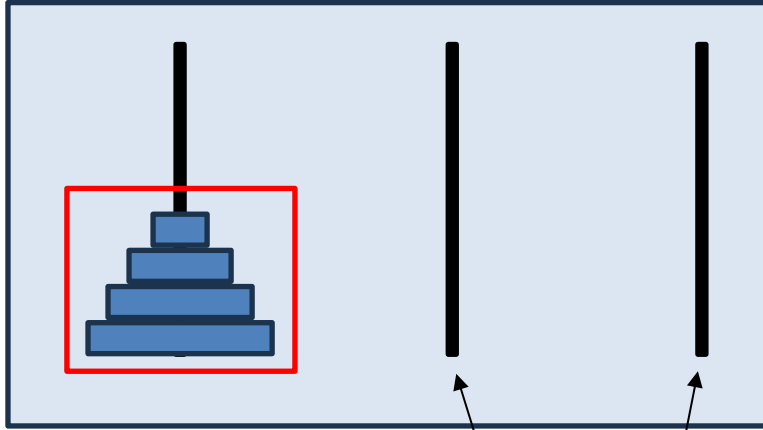
# Towers of Hanoi - Beispiel



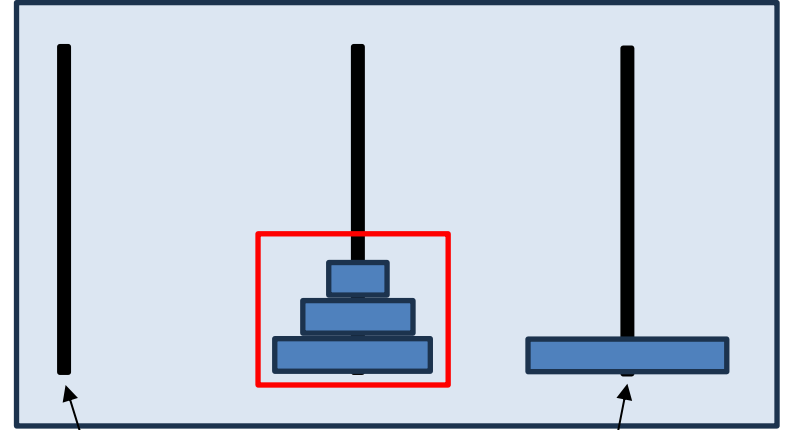
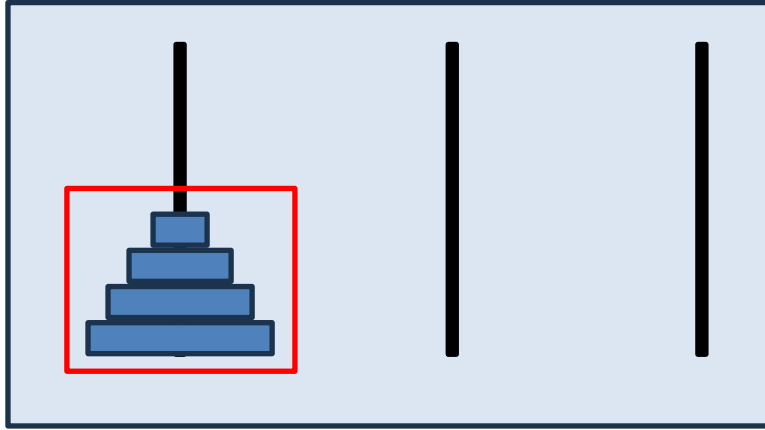


**Was haben die beiden Konstellationen gemeinsam?**

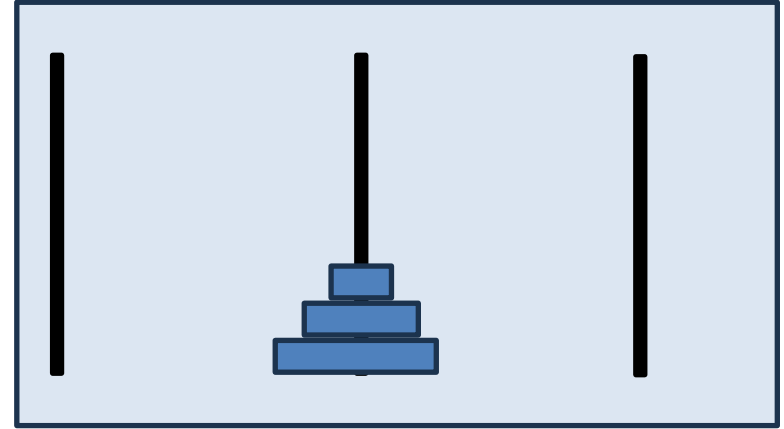
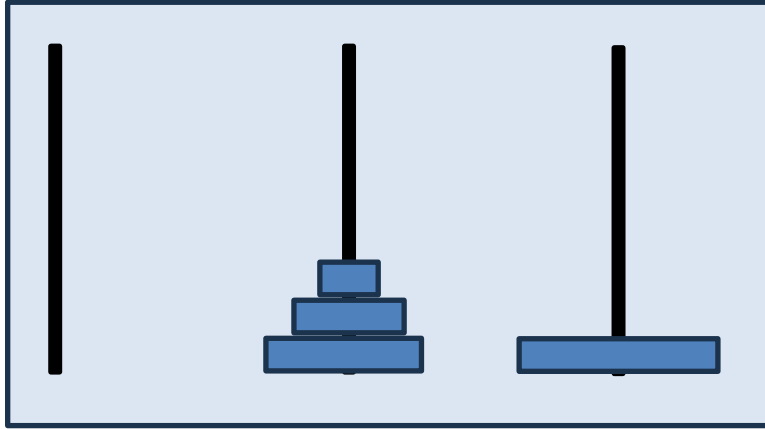




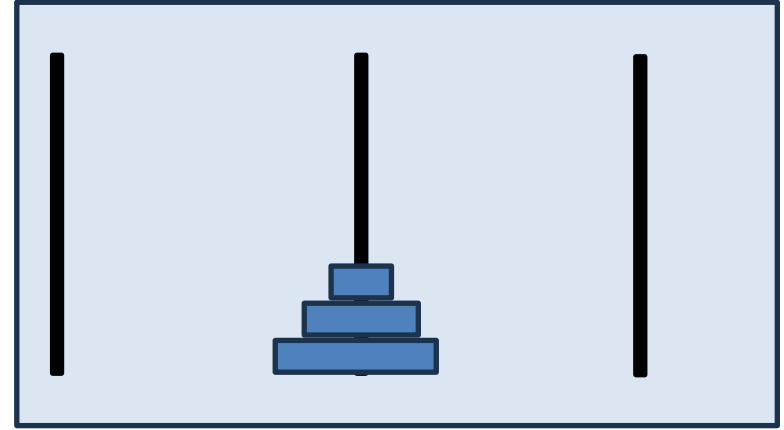
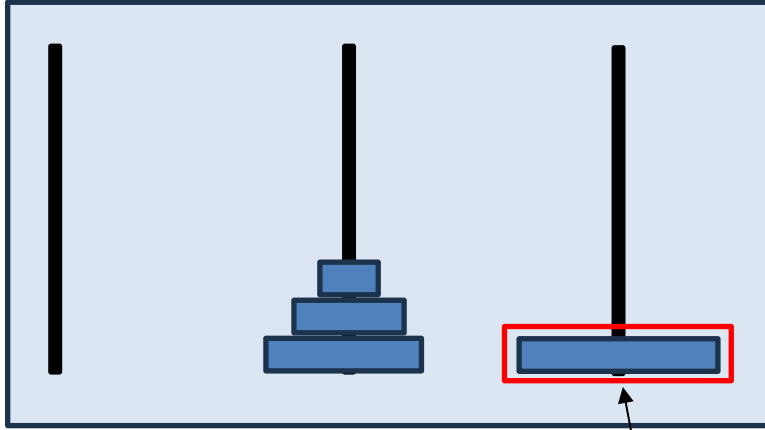
Wir können jede Scheibe in der roten Box auf den verbleibenden Stäben platzieren.



Wir können jede Scheibe in der roten Box auf den verbleibenden Stäben platzieren.

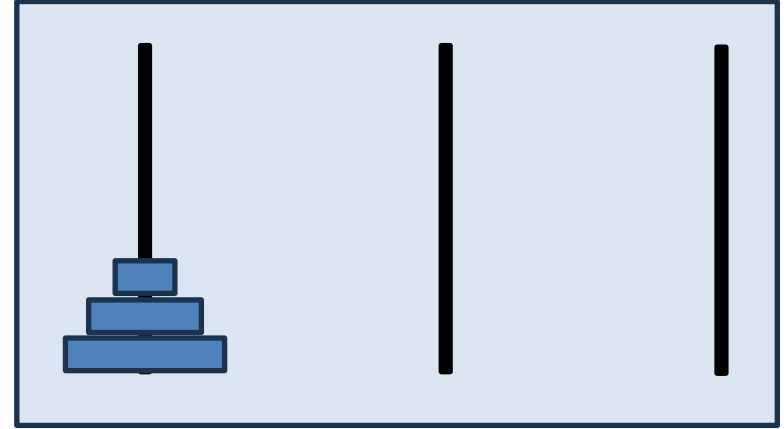
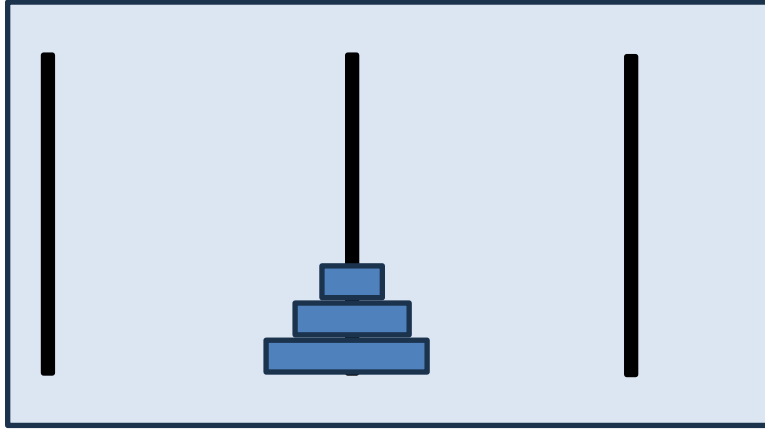


**Können die Konstellationen gleich gelöst werden?**

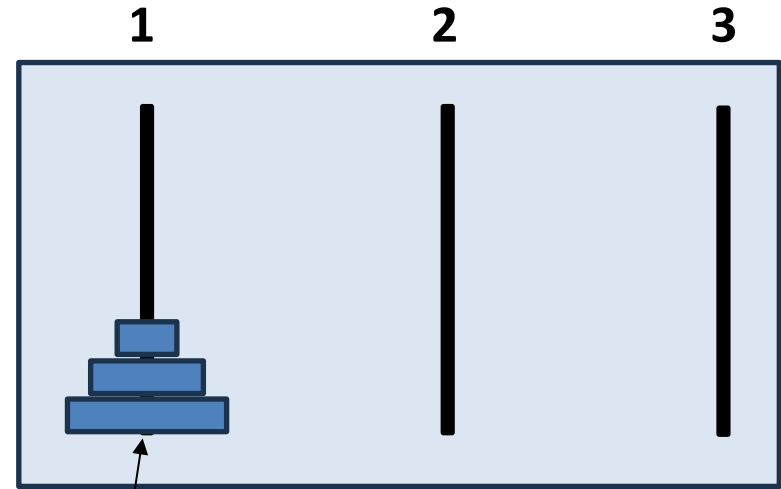
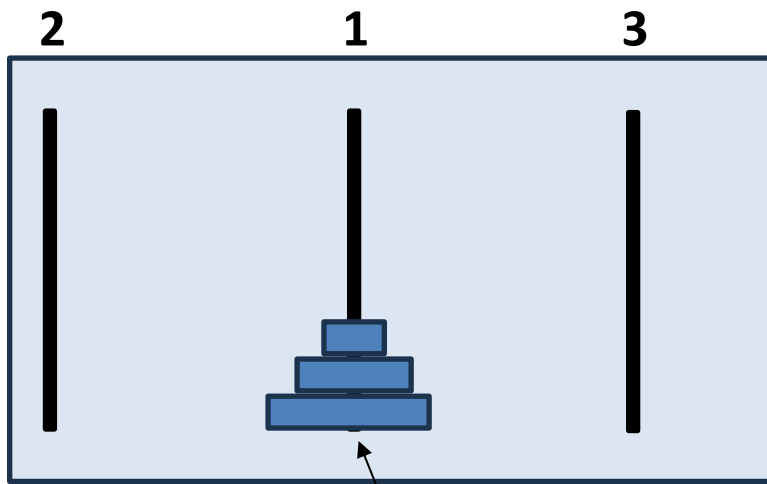


Diese Scheibe ändert das Problem nicht. Es ist die grösste Scheibe und wir können sie deshalb ignorieren und nur die verbleibenden Scheiben betrachten.



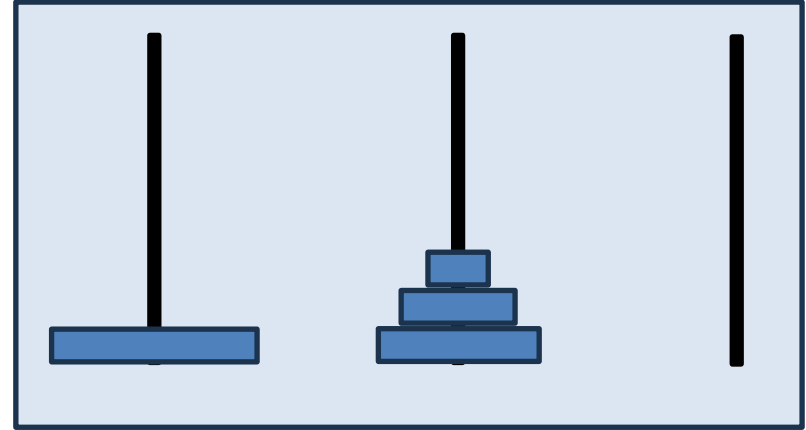
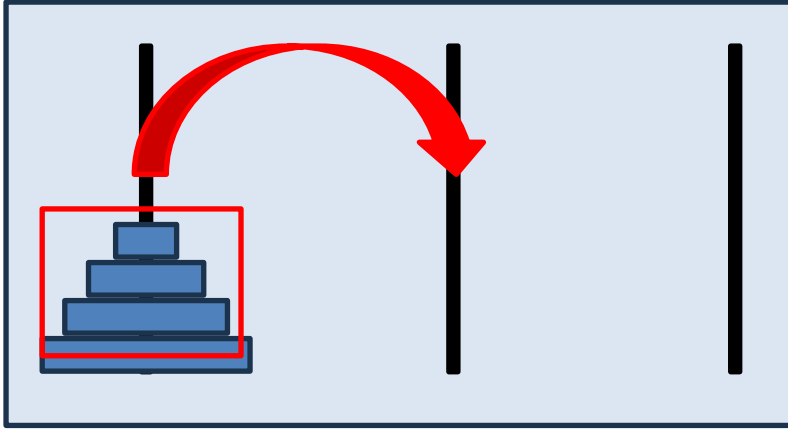


**Können die Konstellationen gleich gelöst werden?**



Das ist das gleiche Problem,  
wenn wir die Stäbe anders  
nummerieren.

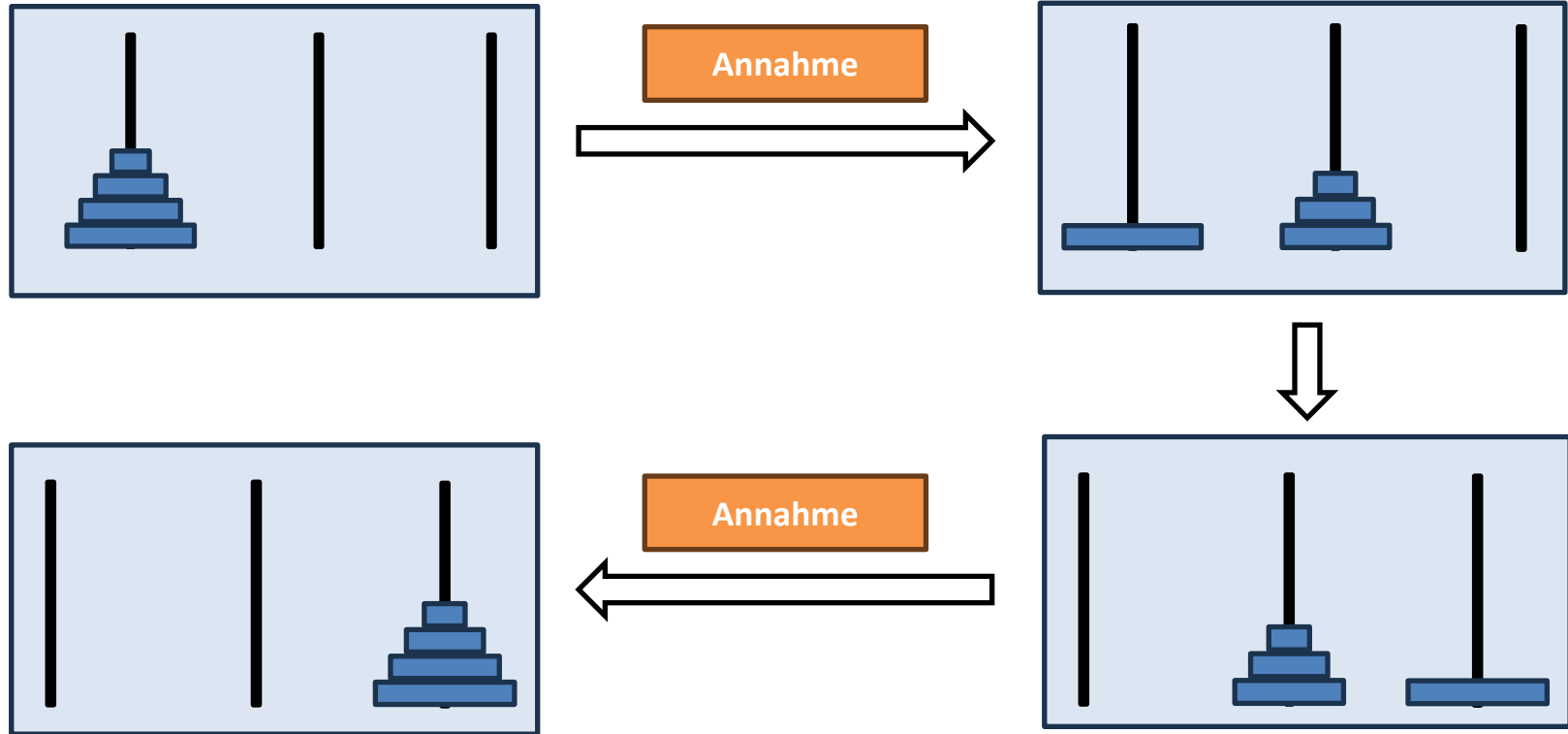




Wir nehmen nun an, wir wissen bereits wie wir einen Stapel von Grösse drei auf einen “freien” Stab bewegen können, wenn wir zwei “freie” Stäbe haben.

**Annahme**

# Wir können das Problem jetzt lösen!





# Stimmt die Annahme?



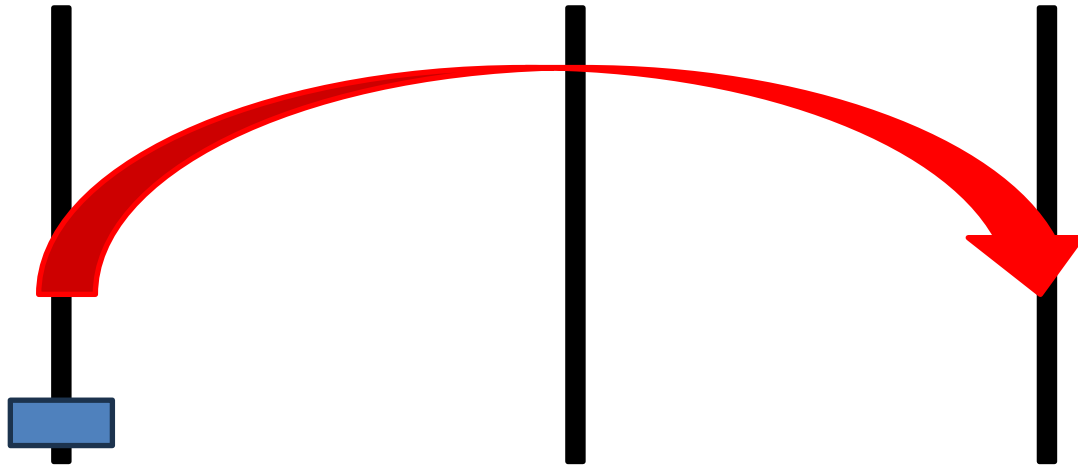
**Nur in einem Fall!**

# Stimmt die Annahme?

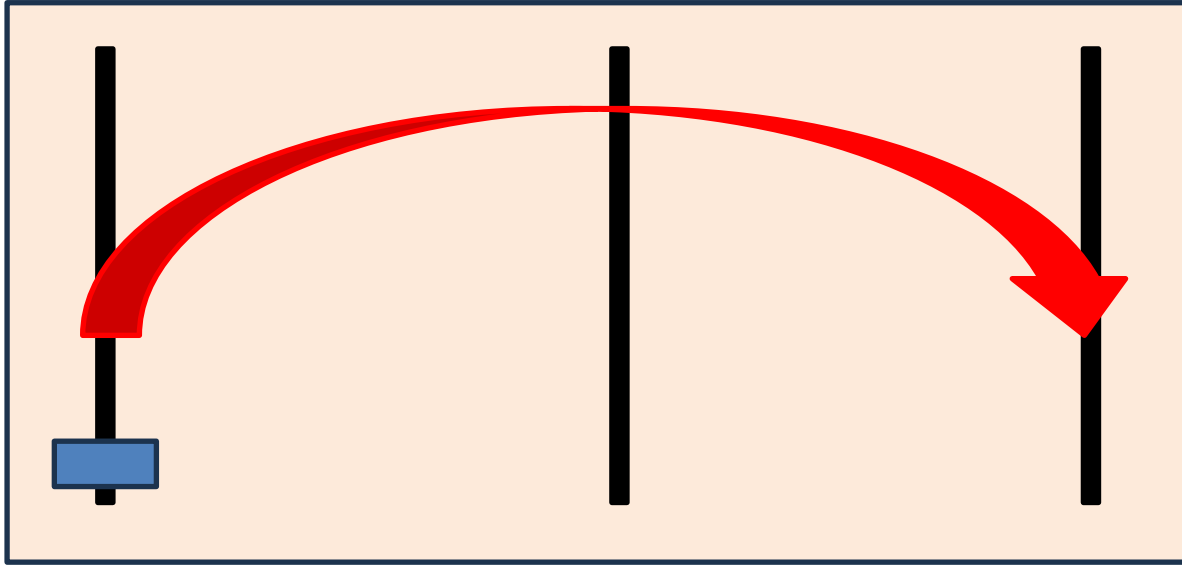


**Das reicht aber!**

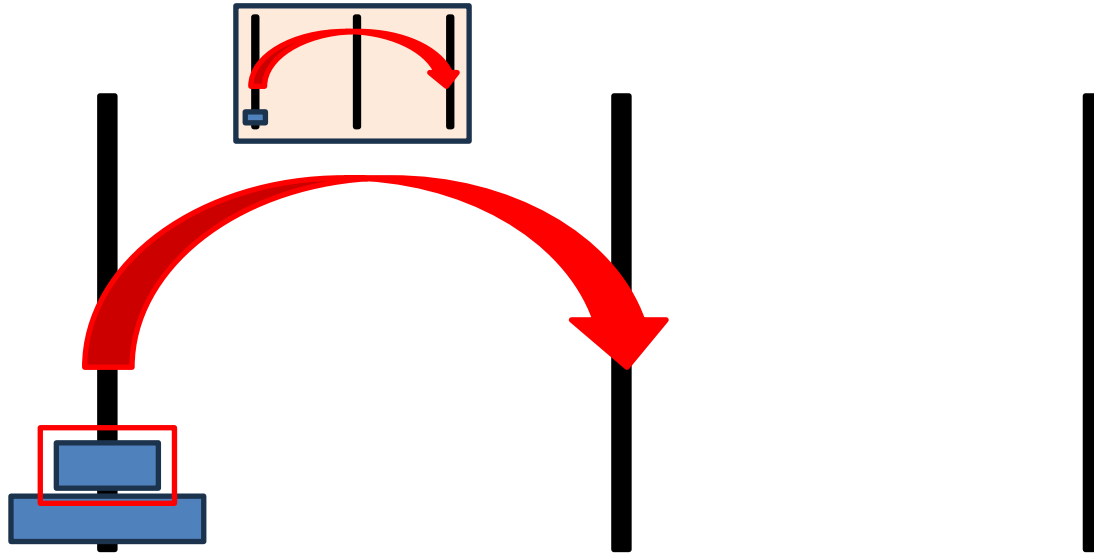
# Towers of Hanoi – Step by Step



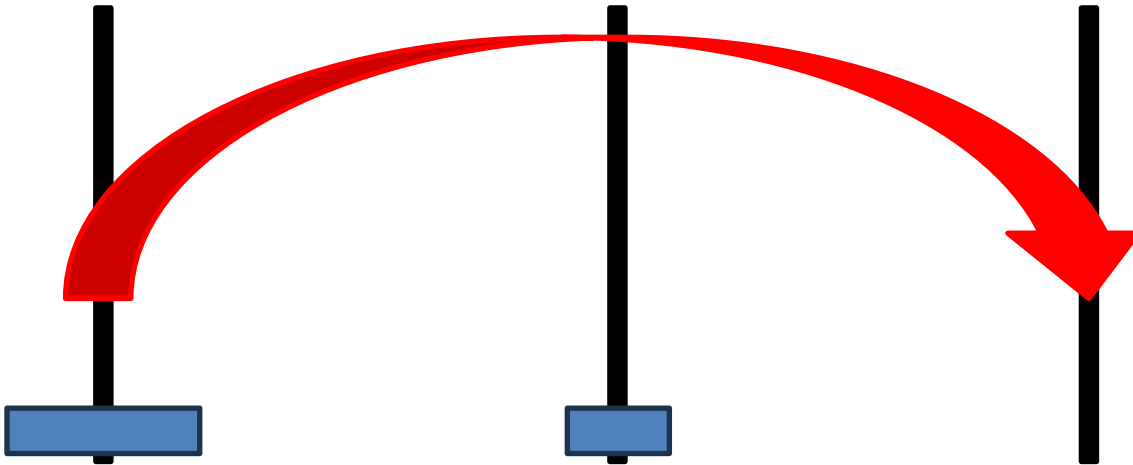
# Towers of Hanoi – Step by Step



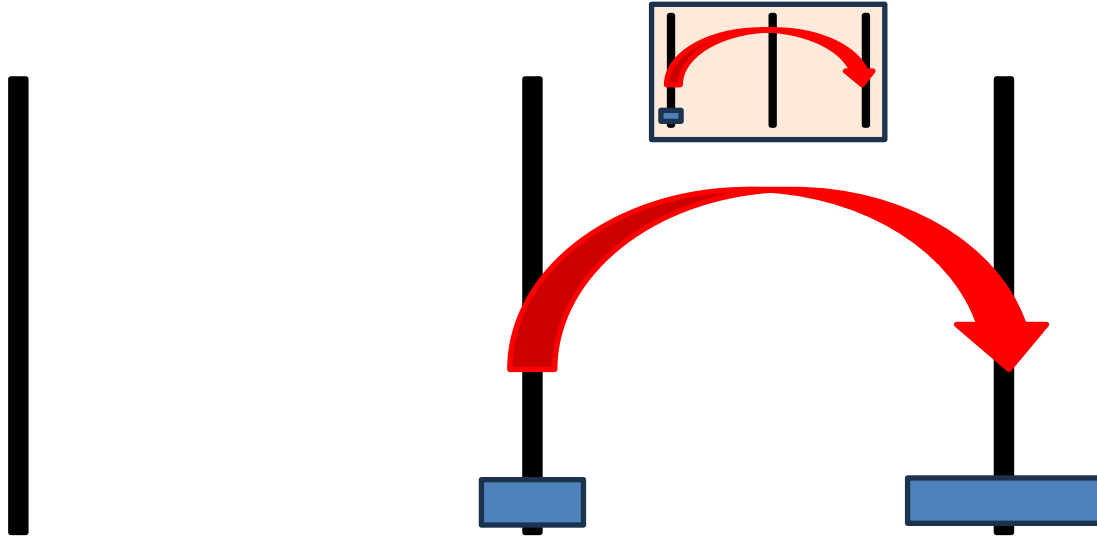
# Towers of Hanoi – Step by Step



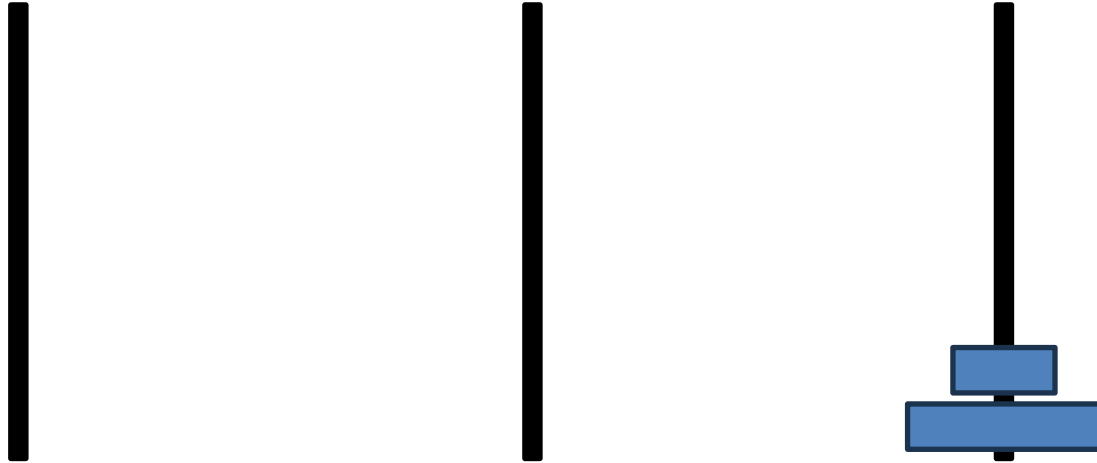
# Towers of Hanoi – Step by Step



# Towers of Hanoi – Step by Step

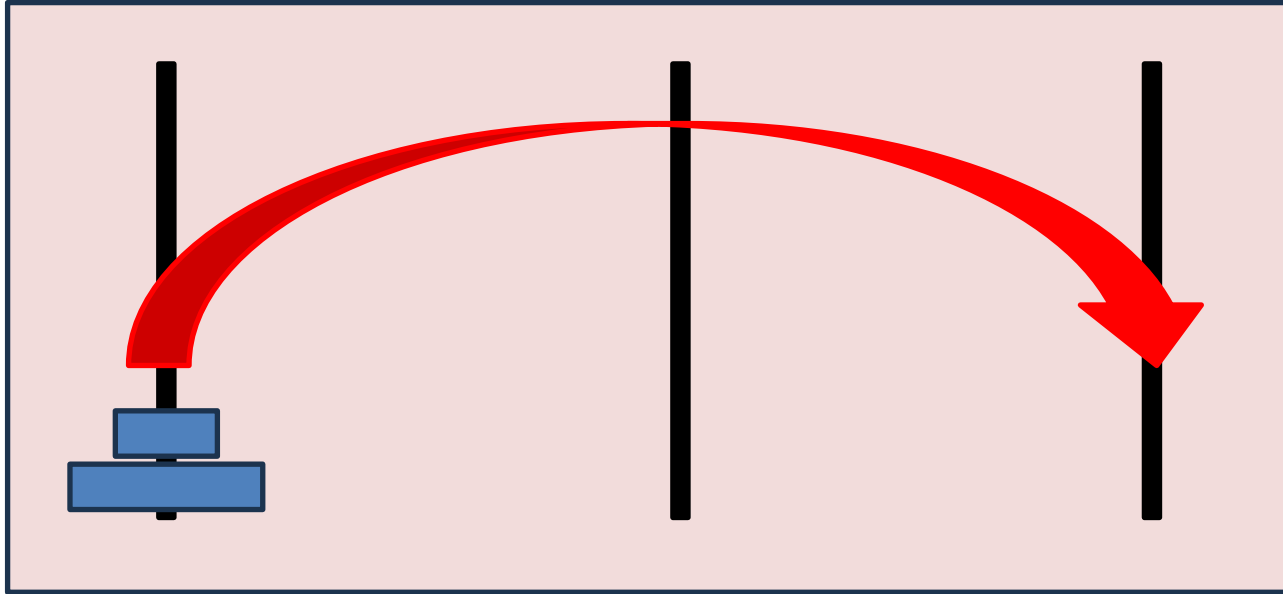


# Towers of Hanoi – Step by Step

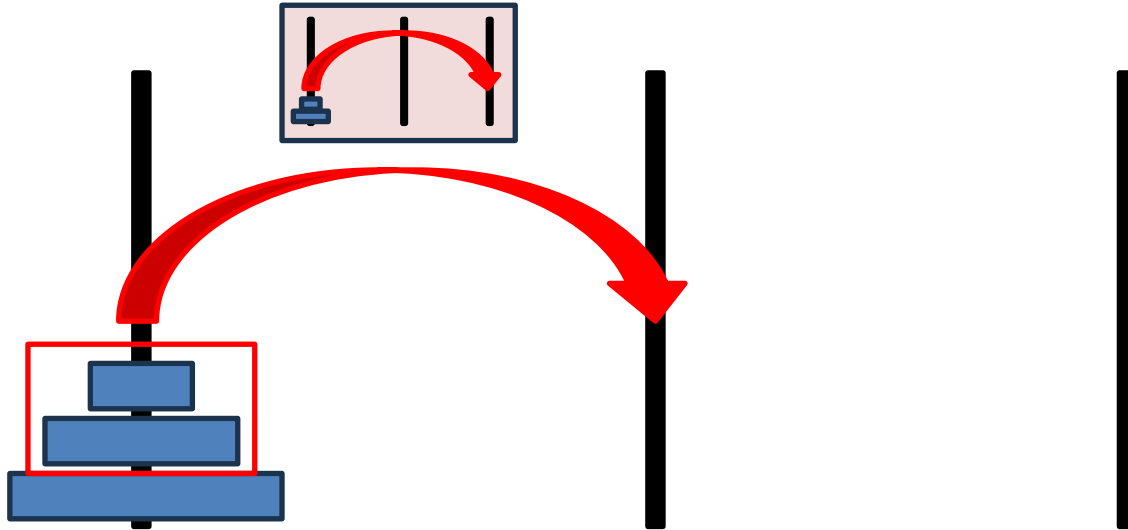




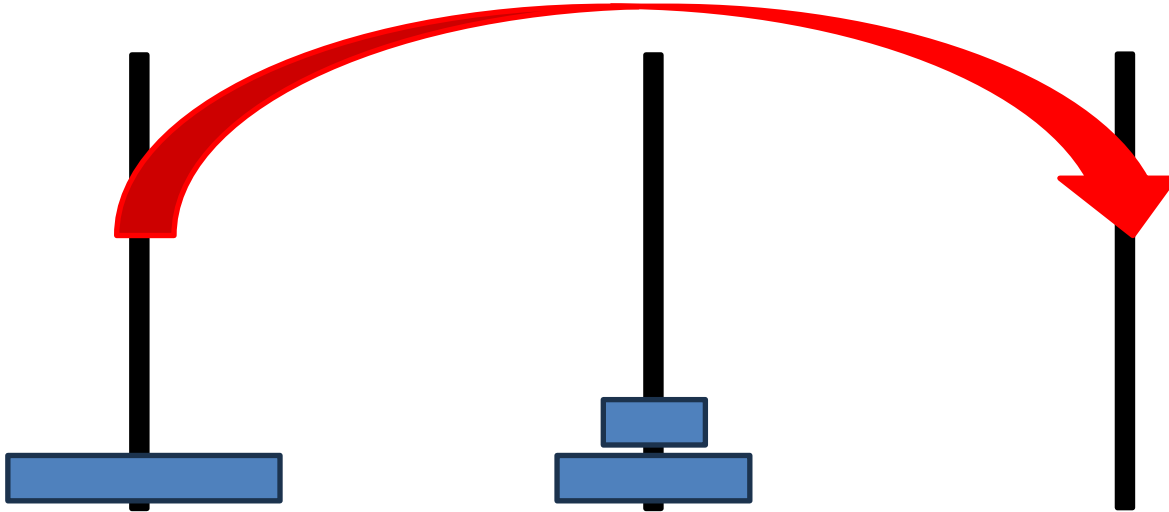
# Towers of Hanoi – Step by Step



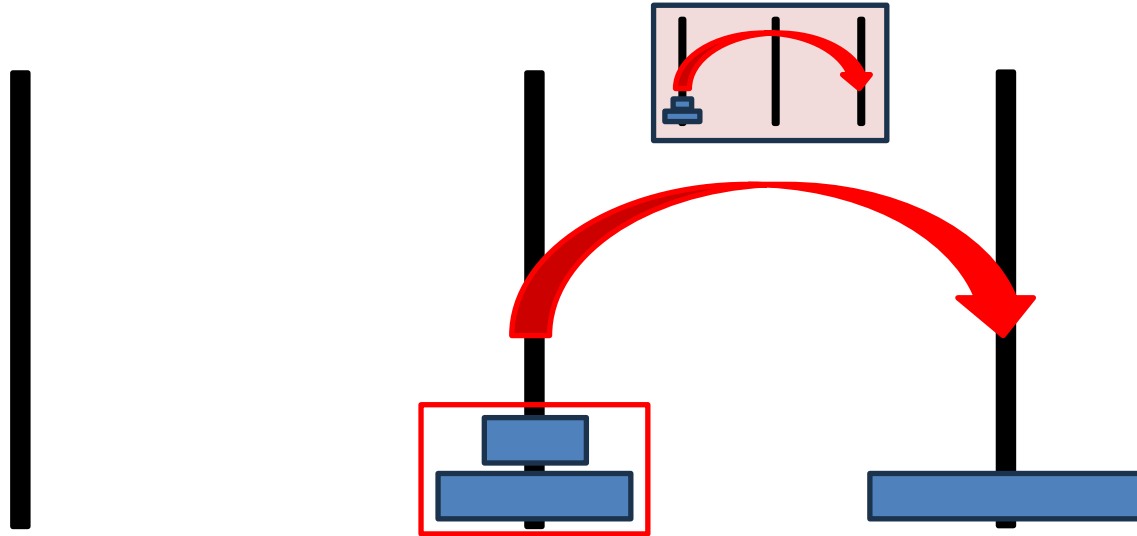
# Towers of Hanoi – Step by Step



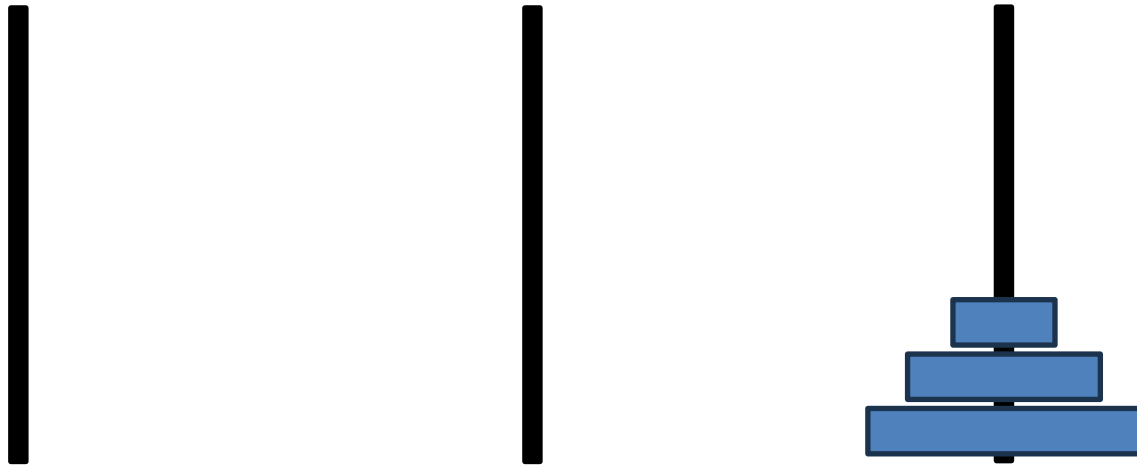
# Towers of Hanoi – Step by Step



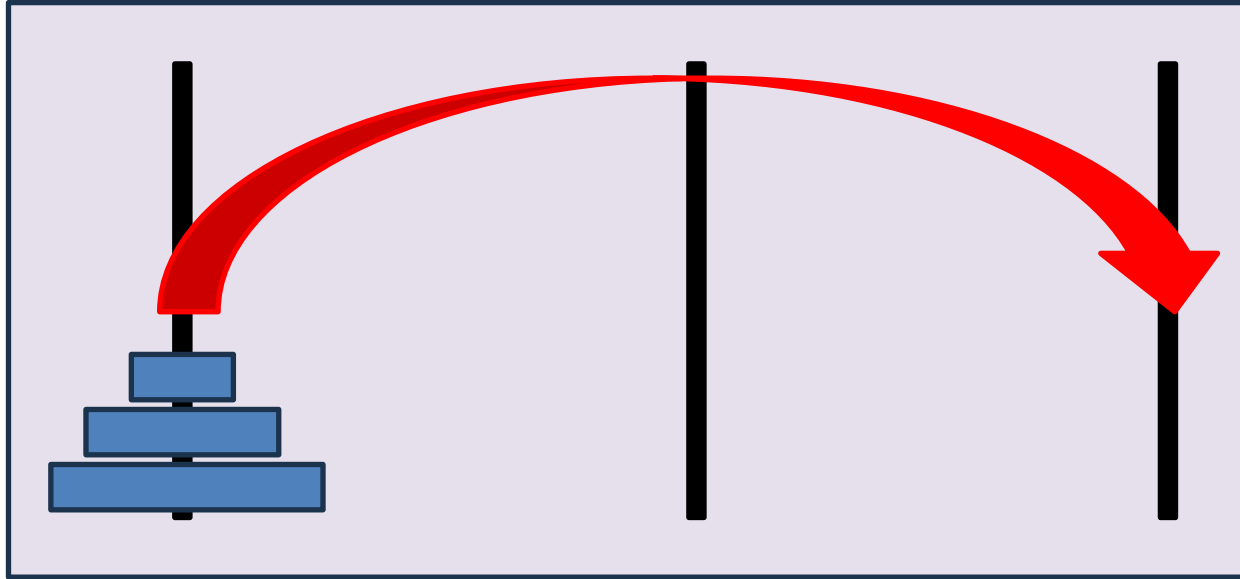
# Towers of Hanoi – Step by Step



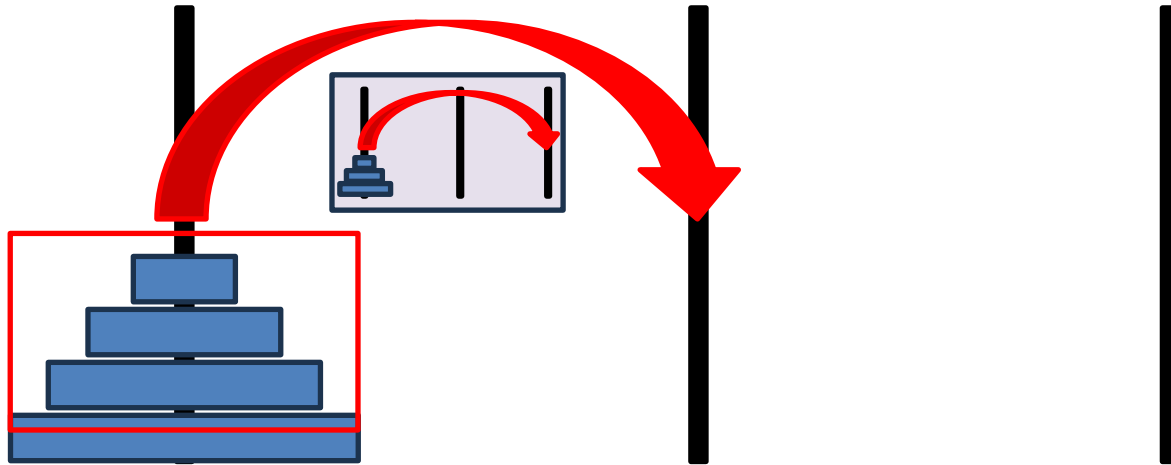
# Towers of Hanoi – Step by Step



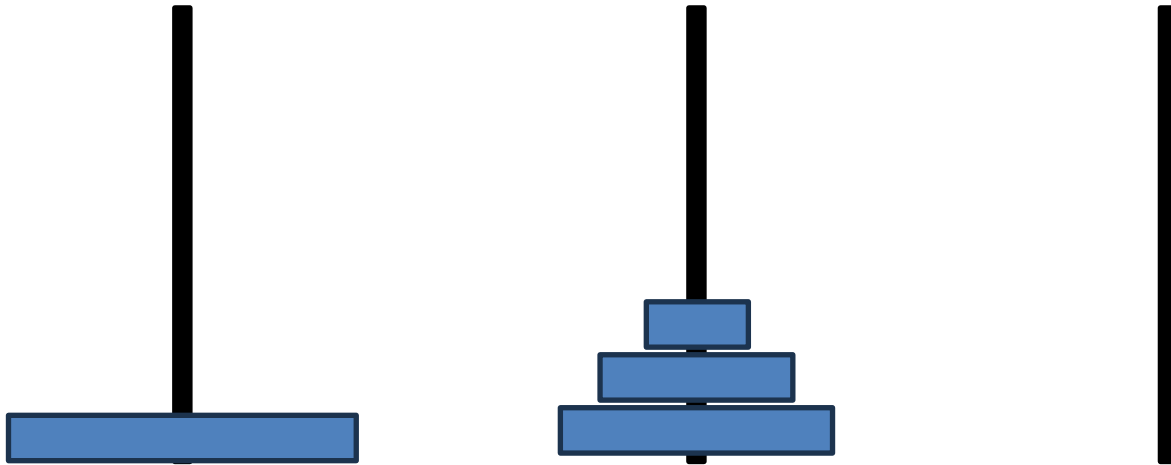
# Towers of Hanoi – Step by Step



# Towers of Hanoi – Step by Step

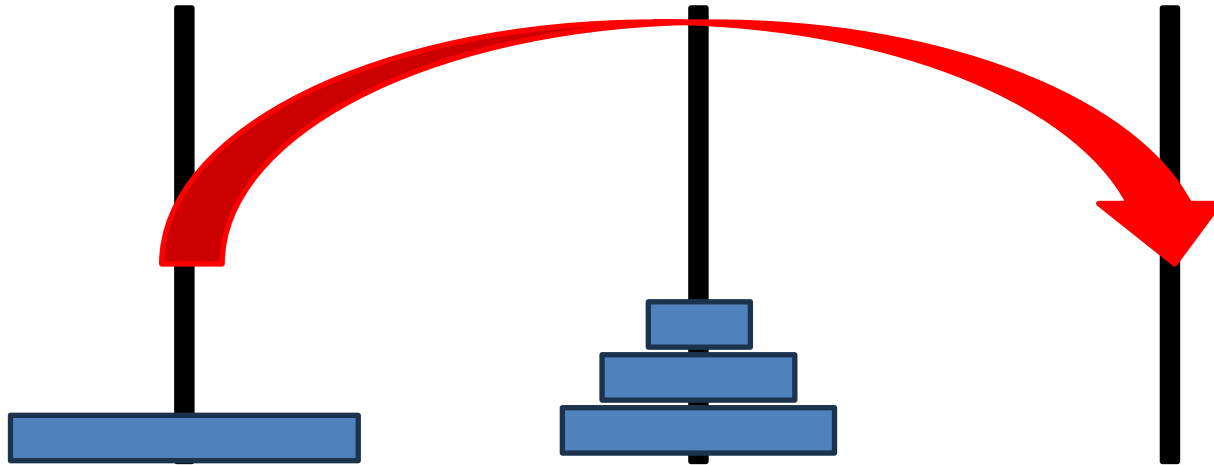


# Towers of Hanoi – Step by Step

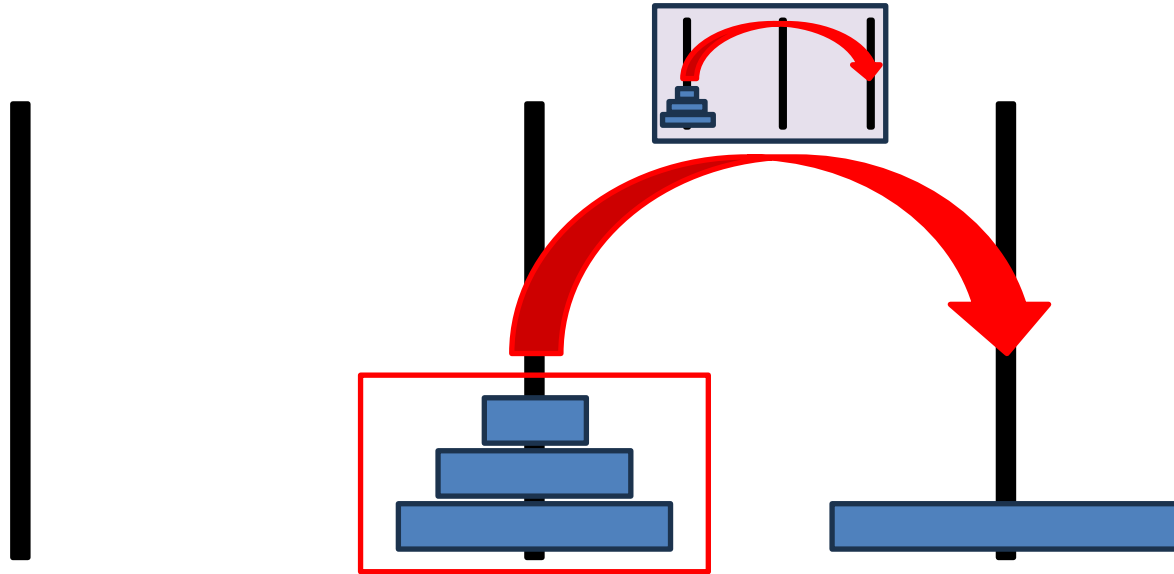




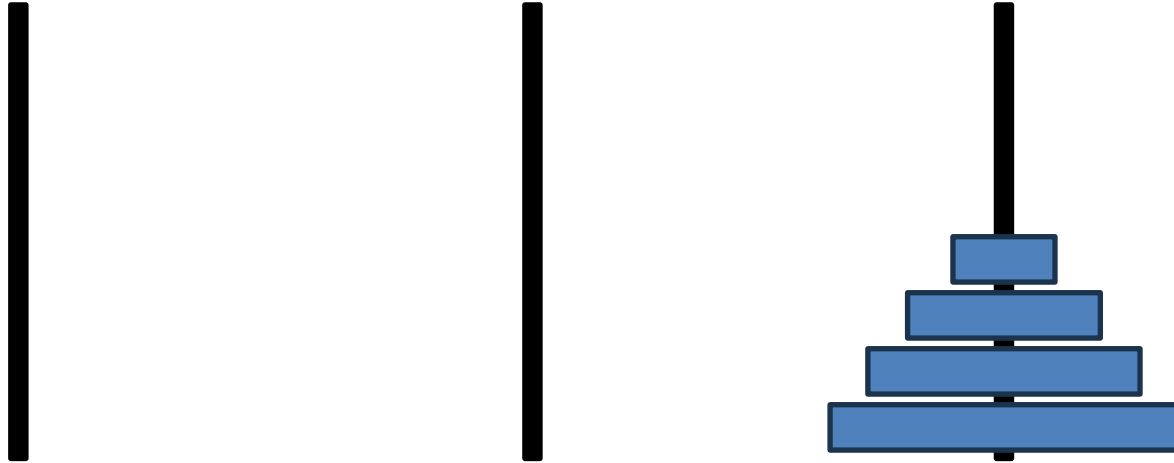
# Towers of Hanoi – Step by Step



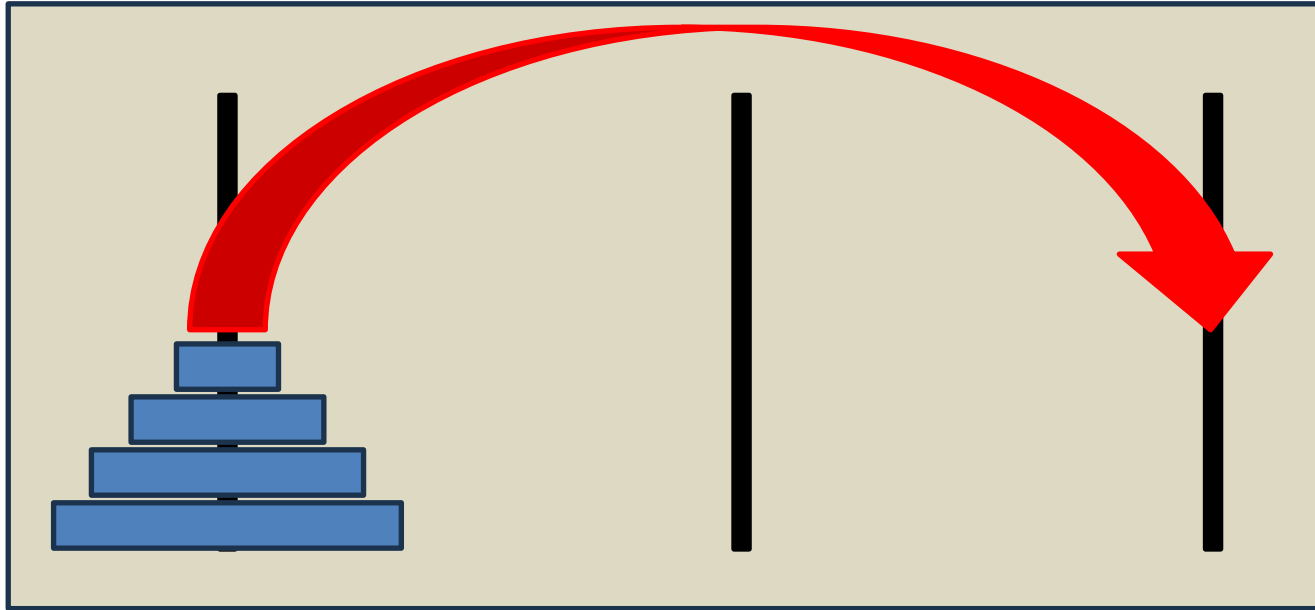
# Towers of Hanoi – Step by Step



# Towers of Hanoi – Step by Step



# Towers of Hanoi – Step by Step



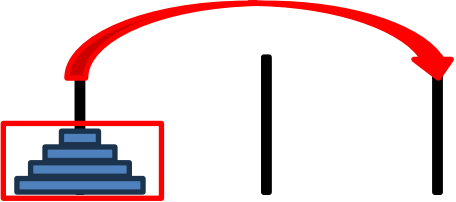
# Towers of Hanoi – Methoden

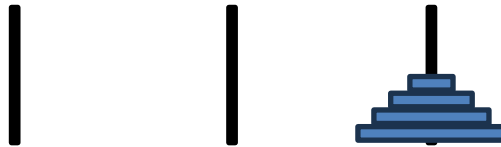
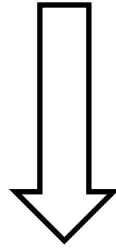
Was soll `solveHanoi(`  `|`  `|` `)` können?

**Input:** Hanoi Konstellation mit zwei “freien” Stäben und ein (Teil-)stapel, der auf einen “freien” Stab bewegt werden soll.

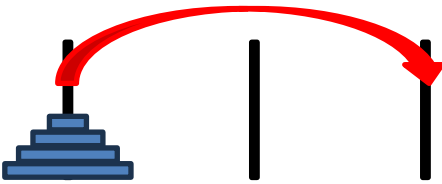
- Ein Stab ist frei, wenn der zu bewegendende Stapel nur kleinere Scheiben enthält, als die Scheiben auf dem Stab.

# Towers of Hanoi – Methoden

`solveHanoi(`  `)`



# Towers of Hanoi – Methoden

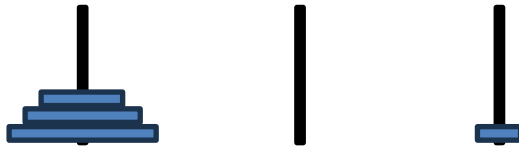
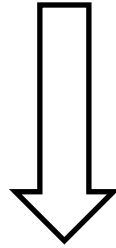
Was soll move(  ) können?

**Input:** Hanoi Konstellation mit mindestens einem “freien” Stab und eine Scheibe soll auf einen “freien” Stab bewegt werden.

- Ein Stab ist frei, wenn der zu bewegendende Stapel nur kleinere Scheiben enthält, als die Scheiben auf dem Stab.

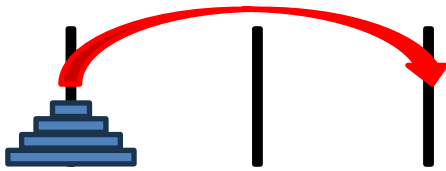
# Towers of Hanoi – Methoden

move(  | | )

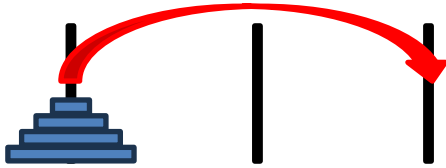
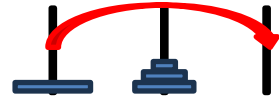




# Towers of Hanoi – Programm

```
solveHanoi( ) {  
    solveHanoi( )  
    move( )  
    solveHanoi( )  
}
```

# Towers of Hanoi – Programm

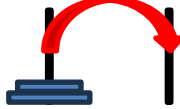
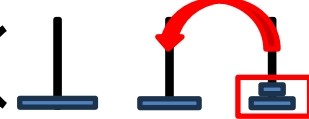
```
solveHanoi( ) {  
    solveHanoi( )  
    move()  
    solveHanoi()  
}
```

Wie ?

# Towers of Hanoi – Programm

```
solveHanoi( | | ) {  
    solveHanoi( | | )  
    move( |   
    solveHanoi(    
}
```

# Towers of Hanoi – Programm

```
solveHanoi(  | | ) {  
    solveHanoi(  | | )  
    move(  )  
    solveHanoi(  )  
}
```

Wie ?

# Towers of Hanoi – Programm

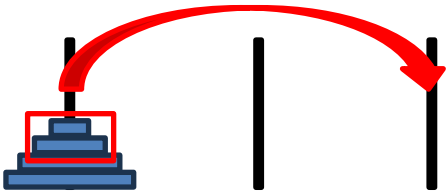
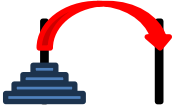
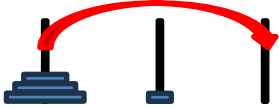
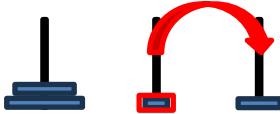
```
solveHanoi( | | ) {  
    solveHanoi( | | )  
    move( | | )  
    solveHanoi(  | )  
}
```

# Towers of Hanoi – Programm

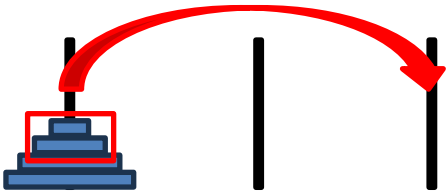
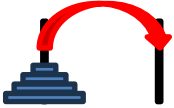
```
solveHanoi( | | ) {  
    solveHanoi( | | )  
    move( | | )  
    solveHanoi(  | )  
}
```

Wie ?

# Towers of Hanoi – Programm

```
solveHanoi( ) {  
    move()  
    move()  
    solveHanoi()  
}
```

# Towers of Hanoi – Programm

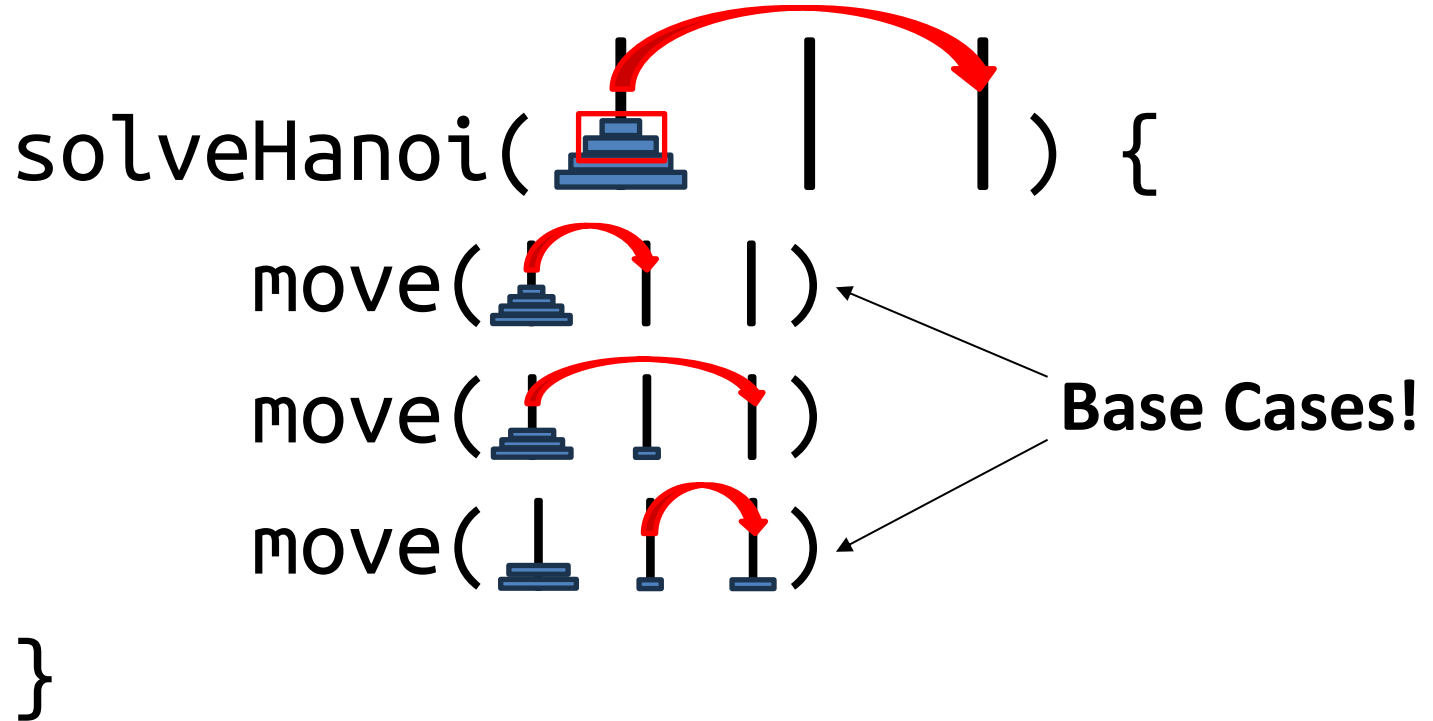
```
solveHanoi( ) {  
    move()  
    move()  
    move()  
}
```



# Towers of Hanoi – Programm

```
solveHanoi( | | ) {  
    move( | | )  
    move( | | )  
    move( | | )  
}
```

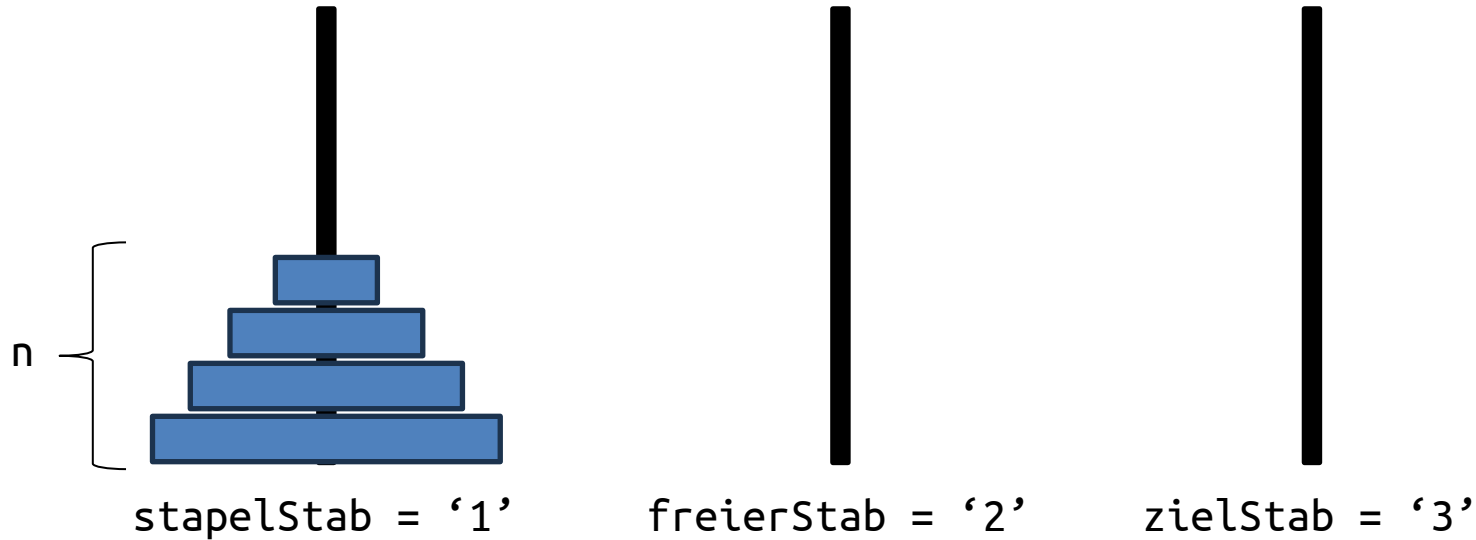
**Base Cases!**



# Towers of Hanoi – Als Programm

- Wir verstehen **wie** man das Problem lösen kann.
  - Das **wie** ist meistens der wichtigste Teil.
- **Zuerst:** Das Problem verstehen.
- **Dann:** Das Programm implementieren.

# Towers of Hanoi – Als Programm



# Towers of Hanoi – Als Programm

- **Methode 1:** `solveHanoi(int n, char stapelStab, char freierStab, char zielStab)`
- **Methode 2:** `move(int n, char stapelStab, char freierStab, char zielStab)`

# Towers of Hanoi – Als Programm

```
1 public class TowerOfHanoi {
2     public static void move(int n, char stapelStab, char freierStab, char zielStab) {
3         System.out.println("Scheibe " + n + " wurde von Stab " + stapelStab + " zu Stab " + zielStab + " bewegt.");
4     }
5     public static void solveHanoi(int n, char stapelStab, char freierStab, char zielStab) {
6         if(n == 1) { // Base Case
7             move(n, stapelStab, freierStab, zielStab);
8         } else { // Step Case
9             solveHanoi(n - 1, stapelStab, zielStab, freierStab);
10            move(n, stapelStab, freierStab, zielStab);
11            solveHanoi(n - 1, freierStab, stapelStab, zielStab);
12        }
13    }
14    public static void main(String[] args) {
15        int scheiben = 4;
16
17        solveHanoi(scheiben, '1', '2', '3');
18    }
19 }
```

# Towers of Hanoi – Als Programm

- **Link zum Code:**  
<https://lec.inf.ethz.ch/infk/eprog/2025/exercises/additionals/TowerOfHanoi.java>



# Prüfungsaufgabe Methodenverständnis FS 2021

Gegeben sei eine Java Methode `test()`, die mit verschiedenen Argumenten für ein Exemplar `s` der Klasse `Suspicious` aufgerufen wird.

```
class Suspicious {  
  
    int i = 0;  
  
    String test(int i) {  
        if (i <= 0) {  
            return "Z";  
        }  
        if (i < 3) {  
            i = i - 1;  
            String result = "T" + test(i) + "T";  
            i = 0;  
            return result;  
        } else if (i < 6) {  
            return "S" + test(i+1) + "S";  
        } else {  
            return "X" + test(i/8);  
        }  
    }  
}
```

Bitte geben Sie für jeden dieser Aufrufe das Resultat an (entweder dem String, der gedruckt wird, oder schreiben Sie "Laufzeitfehler" (oder "exception") in dem Fall dass eine Exception auftritt (den genauen Fehler bzw. die genaue Exception müssen Sie nicht angeben).

```
Suspicious s = new Suspicious();
```

1. `System.out.println(s.test(1));`

**TZT**

---

2. `System.out.println(s.test(3));`

**SSSXZSSS**

---

3. `System.out.println(s.test(9));`

**XTZT**

---

# Notfallaufgabe FS 2021

Gegeben sei eine Methode main in einer Java Klasse.

```
public static void main(String[] args) {  
    /* body */  
}
```

Die folgenden Anweisungen sollen als "Body" (Rumpf) anstelle des Kommentars `/* body */` eingefügt werden und den angegebenen Output produzieren. Bestimmen Sie für jede Anweisung die fehlenden Operatoren so, dass die Anweisung die gezeigte Ausgabe erzeugt. Mögliche Operatoren sind `+`, `-`, `*`, `/` und `%`.

Wenn es für eine Anweisung mehrere mögliche Lösungen gibt, so genügt eine Lösung. Sollte es keine Lösung geben, so schreiben Sie bitte "nicht möglich".

Bereits existierende Operatoren (oder sonstige Teile der Anweisung) dürfen Sie *nicht* verändern. Auch dürfen Sie Klammern weder hinzufügen noch entfernen.



1. `System.out.println(64 / (8 * 2) + (15 - 14) * 2);`

`//Output: 18`

2. `System.out.println(((48 - 8) * (17 - 3) <= 3) || (7 * 0) == 1);`

`//Output: true`

3. `System.out.println("test " + 17 * 17 % 8 + 2);`

`//Output: test 0`